南京大学 NANJING UNIVERSITY

奋进行动 团结奋斗 争先进位 坐言起行 应势而动

第8章 数据库编程

8.1 嵌入式SQL

(2026 春季)

课程内容回顾

❑课程内容回顾

- ➢ 关系模型 & 关系代数
- ➢ 关系数据库系统 & SQL语言
  - 数据定义
  - 数据操纵
  - 数据库安全性
  - 数据库完整性

❑ 本章内容：数据库编程

- ➢ 在应用系统中如何编程访问数据库?
- ➢ SQL语言是关系数据库的标准语言，在应用程序中如何使用SQL语言来访问数据库?

本章学习导引

❑ SQL语言的三种使用方式

- 交互式SQL (Interactive SQL – ISQL)
- 嵌入式SQL (Embedded SQL – ESQL)
- 过程化SQL (Procedural Language SQL – PL/SQL)

	使用方式	应用场景
交互式SQL	命令行 / 批处理	可独立运行，一般供临时用户操作访问数据库用(即席查询, ad-hoc query)
嵌入式SQL	主语言 + ESQL	数据库应用开发
过程化SQL	SQL编程	• 兼有SQL数据访问和高级程序设计语言的流程控制、简单数值处理功能； • 可在数据库服务器中独立运行； • 常用于编写存储过程、存储函数、触发器。

第8章 数据库编程

8.1 嵌入式SQL

8.2 过程化SQL

8.3 存储过程和函数

8.4 ODBC编程

8.5* OLE DB

8.6* JDBC编程

8.7 小结

8.1 嵌入式SQL - 学习导航

- SQL语言提供了两种不同的使用方式
  - 嵌入式SQL (ESQL) & 交互式SQL (ISQL)
- 为什么要引入ESQL?
  - SQL语言是非过程性语言
  - 事务处理应用需要高级语言
- 这两种使用方式细节上有差别，在程序设计的环境下，SQL语句要做某些必要的扩充
  - 如何区分ESQL与用于应用开发的程序设计语言
    - 语句的定界符
    - 主变量 & SQL变量
  - 数据交换
    - SQL通信区，游标(cursor)，select ... into ...
  - 异常处理定义语句：WHENEVER .....
  - 变量赋值，流程控制语句

为什么要引入嵌入式SQL?

- **SQL语言的目标**
  - 为最终用户(end-user)提供存取访问数据库的功能
- **SQL语言的不足**
  - 用户需要知道数据库的模式定义信息：表及其属性定义，视图定义，.....
  - 用户需要掌握复杂的SQL语法
  - 用户在实际使用过程中可能会存在各种误操作，特别是错误的update, delete等数据更新操作
- **解决方案：提供嵌入式SQL**
  - 供应用开发人员(application programmers) 编写数据库应用程序
  - 最终用户(end-user) 不再需要书写SQL语句，而是直接通过应用程序来存取访问数据库

8.1.1 嵌入式SQL的处理过程

8.1.2 嵌入式SQL语句与主语言之间的通信

8.1.3 不用游标的SQL语句

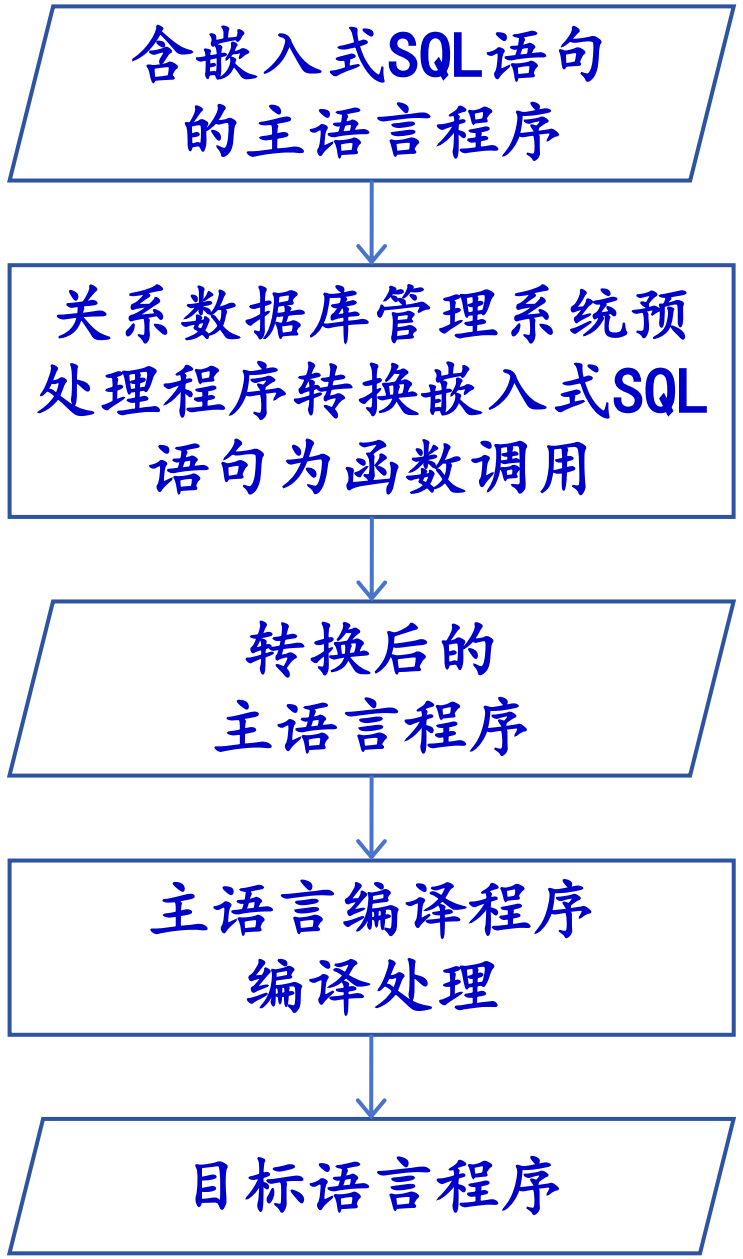
8.1.4 使用游标的SQL语句

8.1.5 动态SQL

8.1.1 嵌入式SQL的处理过程

- 主语言
  - 嵌入式SQL是将SQL语句嵌入程序设计语言中，被嵌入的程序设计语言，如C、C++、Java，称为宿主语言，简称主语言。

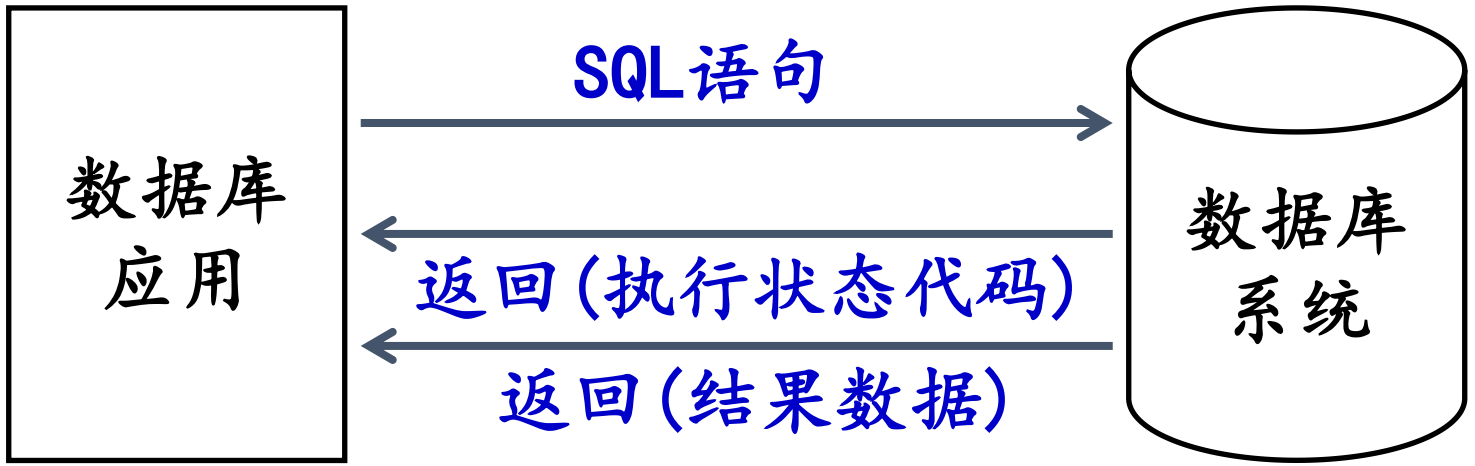
- 处理过程（图8.1）
  - 预编译方法



8.1 嵌入式SQL

- 在使用嵌入式SQL进行数据库应用开发时，需要解决下面四个问题：
  - 主语言语句与嵌入式SQL语句的区别
  - 主语言变量与SQL变量的区别
  - 主语言程序与嵌入式SQL间的通讯
  - SQL查询语句 与 主语言语句 的结果数据交换

- 数据交换模型



主语言语句与嵌入式SQL语句的区别

- （在C语言中）嵌入式SQL语句
  - 带有前缀 'EXEC SQL' 和后缀 ';'
  - 使用 into子句 来获取结果元组值
    （SELECT ... INTO ... 或 FETCH ... INTO ...）
  - 用主变量 ':host_var' 保存查询结果元组中的属性值
    通过前缀 ':' 来区分主变量和SQL语言中的表名或属性名（也被称为SQL变量）

```
EXEC SQL  select  Sno, Sname, Sage
                  into  :hsno, :hsname, :hsage
                  from  Student
                  where  Sno = :givensno ;
```

8.1.1 嵌入式SQL的处理过程

8.1.2 嵌入式SQL语句与主语言之间的通信

8.1.3 不用游标的SQL语句

8.1.4 使用游标的SQL语句

8.1.5 动态SQL

8.1.2 嵌入式SQL语句与主语言之间的通信

- 将SQL嵌入到高级语言中混合编程，程序中会含有两种不同计算模型的语句
  - **SQL语句**：描述性的面向集合的语句，负责操纵数据库
  - **高级语言语句**：过程性的面向记录的语句，负责控制逻辑流程

  它们之间应该如何通信？

- 数据库工作单元与源程序工作单元之间的通信
  - ① 向主语言传递SQL语句的执行状态信息，使主语言能够据此控制程序流程，主要用**SQL通信区**实现
  - ② 主语言向SQL语句提供执行参数，主要用（输入）**主变量**实现
  - ③ 将SQL语句查询数据库的结果交给主语言处理，主要用（输出）**主变量**和**游标**实现

1. SQL通信区

- **SQLCA： SQL Communication Area**
  - SQLCA是一个数据结构

- **SQLCA的用途**
  - SQL语句执行后，数据库系统需要反馈给应用程序的信息，包括
    - 描述系统当前工作状态
    - 描述运行环境
  - 这些信息将送到并保存在SQL通信区中
  - 应用程序从SQL通信区中获取这些状态信息，据此决定接下来执行的语句

SQL通信区（续）

SQLCA使用方法

定义SQLCA

- 用EXEC SQL INCLUDE SQLCA定义

使用SQLCA

- SQLCA中有一个用于存放每次执行SQL语句后返回当前SQL语句执行状态的代码变量SQLCODE；
- 如果SQLCODE等于预定义的常量SUCCESS，则表示SQL语句成功，否则表示出错；
- 应用程序每执行完一条SQL 语句之后都应该测试一下SQLCODE的值，以了解该SQL语句执行情况并做相应处理。

2. 主变量

❑ 主变量（Host Variable）

- ➢ 在嵌入式SQL语句中，可以使用主语言的程序变量来输入或输出数据。
- ➢ 这种可以在SQL语句中使用的主语言程序变量，被简称为主变量。

❑ 主变量的类型

- ➢ 输入主变量
  - 由应用程序对其赋值，在SQL语句引用
- ➢ 输出主变量
  - 由SQL语句对其赋值或设置状态信息，返回给应用程序

```
EXEC SQL SELECT  Sno, Sname, Ssex, Sage, Sdept
        INTO  :Hsno, :Hname, :Hsex, :Hage, :Hdept
        FROM  Student
        WHERE  Sno = :givensno;
```

输出主变量

输入主变量

指示变量

❑ 指示变量（Indicator Variable）

- 是一个整型变量，是一种特殊的‘主变量’，用来“指示”相关主变量的值是否为‘空值’
- 一个主变量可以附带一个指示变量
- 指示变量的用途
  - 指示输入主变量是否为空值
  - 检测输出变量是否为空值，值是否被截断

❑ 指示变量的取值

0	A database value, not null, was assigned to the host variable.
>0	A truncated database character string was assigned to the host variable.
-1	The database value is null, and the host variable value is not a meaningful value.

指示变量的应用示例

[例8.3] 指示变量Gradeid，用于指示查询返回的Grade属性值是否为空

```
EXEC SQL SELECT  Sno, Cno, Grade
        INTO  :Hsno,  :Hcno,  :Hgrade  :Gradeid
        FROM  SC
        WHERE  Sno = :givensno  AND  Cno = :givencno ;
```

[例] 在选课关系SC中，根据输入的课程号将该课程的成绩都置为空值
（在主变量与指示变量之间用 空格 或 保留字 Indicator 相分隔）

```
gra_ind = –1 ;
EXEC  SQL  UPDATE  SC
        SET  Grade = :Hgrade  INDICATOR  :gra_ind
        WHERE  cno = :givencno;
```

主变量&指示变量的使用方法

- 在SQL语句中使用主变量和指示变量的方法
  - 声明（定义）主变量和指示变量

```
BEGIN DECLARE SECTION  /*主变量声明开始 */
    ......    /* 声明主变量和指示变量，方法与普通主语言变量一样 */
END DECLARE SECTION  /*主变量声明结束 */
```

  - 在SQL语句中使用主变量
    - 声明之后的主变量可以在SQL语句中任何一个能够使用表达式的地方出现
    - 为了与数据库对象名（表名、视图名、列名等）区别，SQL语句中的主变量名前要加冒号（:）作为标志
  - 在SQL语句中使用指示变量
    - 指示变量前也必须加冒号标志，且必须紧跟在所指主变量之后，用‘空格’或保留字 Indicator 相分隔
- 在SQL语句之外(主语言语句中)：直接使用主变量和指示变量，不必加冒号前缀

主变量&指示变量的声明

- 为了使用主语言变量，必须首先在DECLARE SECTION部分声明这些变量，Why?

```
exec sql begin declare section;
    char  hsno[10],  givensno[10],  hsname[21];
    int  hsage;
exec sql end declare section;
    ......
EXEC SQL  select  Sno, Sname, Sage
                into  :hsno, :hsname, :hsage
                from  Student
                where  Sno = :givensno ;
```

- [例3.5] 建立“学生”表Student。

```
CREATE TABLE Student (
        Sno   CHAR(9) PRIMARY KEY,
        Sname CHAR(20) UNIQUE,
        Ssex    CHAR(2),
        Sage   SMALLINT,
        Sdept  CHAR(20)
);
```

- 主语言变量声明语句（DECLARE SECTION）作用
  - 在编译时就可以检查主语言变量与其对应属性的数据类型是否一致
  - 为接收从数据库返回的结果值而预先申请内存空间

主变量声明的作用

- 主变量声明的作用 (Declare Section)
  - ① 在编译时对主语言变量与对应属性进行类型一致性检查
  - ② 为接收从数据库返回的结果值而预先申请足够大的内存空间

  - 对于数值类型的属性，常见的主变量类型
    - float cust_discnt;
    - int cust_discnt;

    } 在数据交换时自动进行类型转换

  - 对于字符类型的属性
    - in C: with null terminal character
    - in DB: no terminator, fixed length

主变量声明的示例

```
/* 'begin declare' 和 'end declare' 标志着主变量定义的开始与结束，只有在
   declare section 中定义的主语言变量，才能被用在ESQL语句中，实现应用程序
   与数据库之间的数据交换。*/

exec sql begin declare section;  /* 主变量声明开始 */

/* 字符类型的主变量，需要预先分配好足够大的内存空间（至少比对应的表中属性
   的最大长度+1，避免在返回查询结果时出现内存溢出错误。*/
    char  Deptname[21];
    char  Hsno[10];
    char  Hsname[21];
    char  Hssex[3];
    int  HSage;
    int  NEWAGE;

exec sql end declare section; /* 主变量声明结束 */
```

3. 游标

❑ 为什么要使用游标?

- SQL语言与主语言具有不同数据处理方式：非过程化 vs 面向过程。
- 不同数据处理对象：SQL语言是面向集合的，一条SQL语句原则上可以产生或处理多条记录；主语言是面向记录的，一组主变量一次只能存放一条记录。
- 仅使用主变量并不能完全满足SQL语句向应用程序输出数据的要求
- 嵌入式SQL引入了游标的概念，可用来协调这两种不同的处理方式

❑ 游标（Cursor）

- 游标是系统为用户开设的一个数据缓冲区，用于存放SQL查询的执行结果
- 每个游标都有一个名字 -- 游标名
- 当游标被打开后，用户可以用SQL语句逐一从游标中获取结果记录，并赋给主变量，交由主语言进一步处理

4. 建立和关闭数据库连接

（1）建立数据库连接

EXEC SQL CONNECT TO target [AS connection-name] [USER user-name];

- target是要连接的数据库服务器
  - 可以是一个常见的服务器标识串，如 <dbname>@<hostname>:<port>
  - 可以是包含服务器标识的SQL串常量
  - 也可以是 DEFAULT
- connect-name是可选的连接名，连接名必须是一个有效的标识符
- 在整个程序内只有一个连接时可以不指定连接名
- 程序运行过程中可以修改当前连接

  EXEC SQL SET CONNECTION connection-name | DEFAULT;

（2）关闭数据库连接

EXEC SQL DISCONNECT [connection-name];

5. 程序实例

- [例8.1] 依次检查某个系的学生记录，交互式更新某些学生年龄。

```
EXEC SQL BEGIN DECLARE SECTION; /*主变量声明开始*/
    char  Deptname[21];
    char  Hsno[10];
    char  Hsname[21];
    char  Hssex[3];
    int  HSage;
    int  NEWAGE;
EXEC SQL END DECLARE SECTION; /*主变量声明结束*/

long SQLCODE;
EXEC SQL INCLUDE SQLCA; /*定义SQL通信区*/
```

程序实例（续）

```
int main(void) /*C语言主程序开始*/
{
    int count = 0;
    char yn; /*变量yn代表yes或no*/
    printf("Please choose the department name(CS/MA/IS): ");
    scanf("%s",deptname); /*为主变量deptname赋值*/
    EXEC SQL CONNECT TO TEST@localhost:54321 USER
            "SYSTEM"/"MANAGER"; /*连接数据库TEST*/
    EXEC SQL DECLARE SX CURSOR FOR /*定义游标SX*/
            SELECT Sno,Sname,Ssex,Sage /*游标SX对应的查询语句*/
            FROM Student
            WHERE SDept = :deptname;
    EXEC SQL OPEN SX;
        /*打开游标SX，执行游标对应的查询，并指向查询结果的第一行*/
```

程序实例（续）

```
for ( ; ; ) /*用循环结构逐条处理结果集中的记录*/
{
    EXEC SQL FETCH SX INTO :HSno,:Hsname,:HSsex,:HSage;
            /*推进游标，将当前数据放入主变量*/
    if (SQLCA.SQLCODE != 0) /*SQLCODE != 0，表示操作不成功*/
        break; /*利用SQLCA中的状态信息决定何时退出循环*/
    if (count++ == 0) /*如果是第一行的话，先打出行头*/
        printf("\n%-10s %-20s %-10s %-10s\n", "Sno","Sname","Ssex","Sage");
    printf("%-10s %-20s %-10s %-10d\n ", HSno,Hsname,Hssex,HSage);
            /*打印查询结果*/
    printf( “UPDATE AGE(y/n)?” ); /*询问用户是否要更新该学生的年龄*/
    do {
        scanf("%c",&yn);
    } while(yn != 'N' && yn != 'n' && yn != 'Y' && yn != 'y');
```

程序实例（续）

```
        if (yn == 'y' || yn == 'Y') /*如果选择更新操作*/
        {
            printf("INPUT NEW AGE:");
            scanf("%d",&NEWAGE); /*用户输入新年龄到主变量中*/
            EXEC SQL UPDATE Student /*嵌入式SQL更新语句*/
                SET Sage = :NEWAGE
                WHERE CURRENT OF SX;
        } /*对当前游标指向的学生年龄进行更新*/
    }
    EXEC SQL CLOSE SX; /*关闭游标SX，不再和查询结果对应*/
    EXEC SQL COMMIT WORK; /*提交更新*/
    EXEC SQL DISCONNECT TEST; /*断开数据库连接*/
}
```

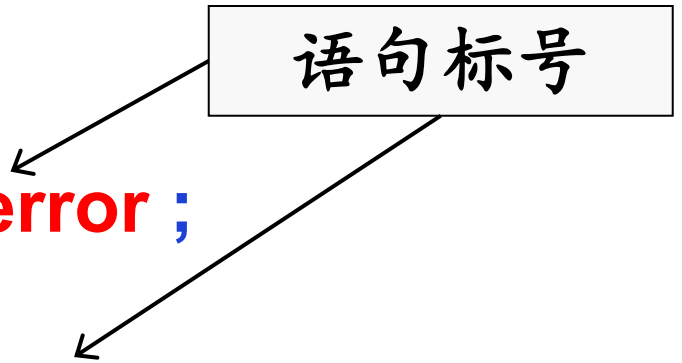
程序实例（续）

- [思考] 在例8.1的嵌入式SQL程序中，还存在什么BUG？(P248~249)
  - 只在 EXEC SQL FETCH SX INTO ..... 语句之后检查了状态码 SQLCA.SQLCODE
  - 在其他嵌入式SQL语句之后，都没有检查SQL语句的执行状态码！

- 解决方案
  - 在程序中添加关于异常处理(Condition Handling)的规则定义

异常处理（Condition Handling）

- whenever 语句用于定义异常处理办法
- 最常见的执行异常是：sqlerror 和 not found
- 最常用的异常处理办法是：
  - 用goto语句跳转到特定的语句标号处，进行异常处理
  - ……




```
exec sql whenever sqlerror goto report_error ;

exec sql whenever not found goto notfound ;
```

异常处理（续）

❑ Whenever 语句

```
EXEC SQL WHENEVER  condition  action;
```

- 为SQL语句的执行设置一个‘异常捕捉’机制
- 预编译器在处理过程中，会根据WHENEVER语句的定义，在每一条嵌入式SQL语句之后添加下面的异常状态检查命令：

```
if ( condition ) { action }
```

异常处理（续）

EXEC SQL WHENEVER condition action;

❑ CONDITIONS

① SQLERROR

- 严重的SQL语句执行错误，出现这类异常系统将终止应用程序的执行。

② NOT FOUND

- SQL语句得到成功执行，但并没有访问到任何元组（Select, Fetch, Insert, Update, or Delete等数据操纵语句）
- 捕捉这类异常，通常用于结束循环（如FOR、WHILE等循环），或者改变程序的控制流程（如用于IF、SWITCH等语句）

③ SQLWARNING

- 非严重但值得注意的异常，通常不会影响到应用程序的执行
- 相关的warning信息被保存在SQL通讯区 SQLCA 中；

异常处理（续）

EXEC SQL WHENEVER condition action;

- **ACTIONS**
  - ① **CONTINUE**
    - 忽略相关的异常，不改变程序的控制流程
    - 也可用于撤消之前关于**condition**的异常处理定义
  - ② **GOTO label**
    - 应用程序将跳转到标号**label**指定处，继续执行
  - ③ **STOP**
    - 立即终止应用程序的执行，同时回滚(rollback)当前事务并撤消与数据库的连接(disconnect)
  - ④ **DO function | BREAK | CONTINUE**
    - 可以用**DO function**调用一个 C语言函数，函数执行结束后，控制将返回触发该异常的SQL语句之后继续执行；也可以用**BREAK**或**CONTINUE**来改变触发该异常的SQL语句所在循环体的执行路径。

交互式访问数据库的程序段

```
exec sql whenever sqlerror goto report_error;
exec sql whenever not found goto finish;

while (prompt(cid_prompt, 1, cust_id, 4) >= 0)
{
   exec  sql  select  cname,  discnt
                  into  :cust_name, :cust_discnt
                  from  customers
                  where  cid = :cust_id;
   exec sql commit work;

   printf("CUSTOMER'S NAME IS  %s AND DISCNT IS  %5.1f\n",
                  cust_name, cust_discnt);
   continue;

notfound:
   printf("Can't find customer %s, continuing\n", cust_id);
}
```

/* whenever 异常处理的定义 */

/* 用prompt函数获取用户输入的客户编号，并保存在主变量cust_id中 */

/* 通过into子句实现DBMS与应用程序之间的数据交换(单条结果元组) */

程序实例（续）

- 使用嵌入式SQL编写的数据库应用程序，如果只考虑对数据库访问部分，其程序结构大致如下：

① **The Declare Section**
- 声明主变量、指示变量、游标

② **Condition Handling**
- 异常处理规则定义

③ **SQL Connect to Database**
- 建立数据库连接

④ **Main Body of Application Program**
- 用主语言编写的应用程序，包括应用界面和数据处理逻辑的实现
- 用ESQL编写的数据库访问语句

⑤ **SQL Disconnect**
- 撤消与数据库的连接

8.1.1 嵌入式SQL的处理过程

8.1.2 嵌入式SQL语句与主语言之间的通信

8.1.3 不用游标的SQL语句

8.1.4 使用游标的SQL语句

8.1.5 动态SQL

8.1.3 不用游标的SQL语句

- 不用游标的SQL语句的种类
  - 说明性语句
  - 数据定义语句
  - 数据控制语句
  - 查询结果为单记录的SELECT语句
  - 非CURRENT形式的INSERT、DELETE、UPDATE等增删改语句

1. 查询结果为单记录的SELECT语句

- 这类语句不需要使用游标，只需用INTO子句指定存放查询结果的主变量。
- [例8.2] 根据学生号码查询学生信息。

```
/* 首先，将要查询的学生的学号赋给主变量givensno，然后执行下述查询 */
EXEC SQL SELECT  Sno, Sname, Ssex, Sage, Sdept
         INTO  :Hsno, :Hname, :Hsex, :Hage, :Hdept
         FROM  Student
         WHERE  Sno = :givensno;
/* 如果执行成功，结果元组被保存在主变量Ssno、Hname.....中 */
```

- 说明：
  - INTO子句、WHERE子句和HAVING短语的条件表达式中均可以使用主变量
  - 查询返回的记录中，可能某些列为空值NULL
  - 如果查询结果实际上并不是单条记录，而是多条记录，则程序出错，关系数据库管理系统会在SQLCA中返回错误信息

1. 查询结果为单记录的SELECT语句（续）

- 指示变量的使用
- [例8.3] 查询某个学生选修某门课程的成绩。假设已经把将要查询的学生的学号赋给了主变量givensno，将课程号赋给了主变量givencno。

```
EXEC SQL SELECT Sno,Cno,Grade
        INTO  :Hsno, :Hcno, :Hgrade :Gradeid    /* 指示变量 Gradeid */
        FROM  SC
        WHERE Sno = :givensno  AND  Cno = :givencno;
IF (Gradeid < 0) {
    ..... /* 学生givensno在课程givencno上的成绩为空值，数据库系统不会在主
             变量Hgrade中写入任何值 */
} ELSE {
    ..... /* 学生givensno在课程givencno上的成绩不为空，且保存在主变量
             Hgrade中 */
}
```

2. 非CURRENT形式的增删改语句

- 在UPDATE的SET子句和WHERE子句中可以使用主变量，SET子句还可以使用指示变量

- [例8.4] 修改某个学生选修1号课程的成绩。

```
/* 需修改成绩的学生的学号已赋给主变量 givensno */
/* 新的成绩已赋给主变量 newgrade */
EXEC SQL  UPDATE SC
        SET  Grade = :newgrade
        WHERE  Sno = :givensno  and  Cno = '1' ;
```

2. 非CURRENT形式的增删改语句（续）

- [例8.5] 某个学生新选修了某门课程，将有关记录插入SC表中。假设插入的学号已赋给主变量stdno，课程号已赋给主变量couno。

```
/* 由于该学生刚选修课程，成绩应为空，所以要把成绩指示变量
   gradeid赋值为 -1 */
gradeid = -1;  /* gradeid为指示变量 */
EXEC SQL  INSERT
        INTO  SC(Sno, Cno, Grade)
        VALUES(:stdno, :couno, :gr  :gradeid);
            /* stdno、couno、gr为主变量 */
```

- [思考] 不使用指示变量，如何插入一条成绩为空的选课记录？

8.1.1 嵌入式SQL的处理过程

8.1.2 嵌入式SQL语句与主语言之间的通信

8.1.3 不用游标的SQL语句

8.1.4 使用游标的SQL语句

8.1.5 动态SQL

8.1.4 使用游标的SQL语句

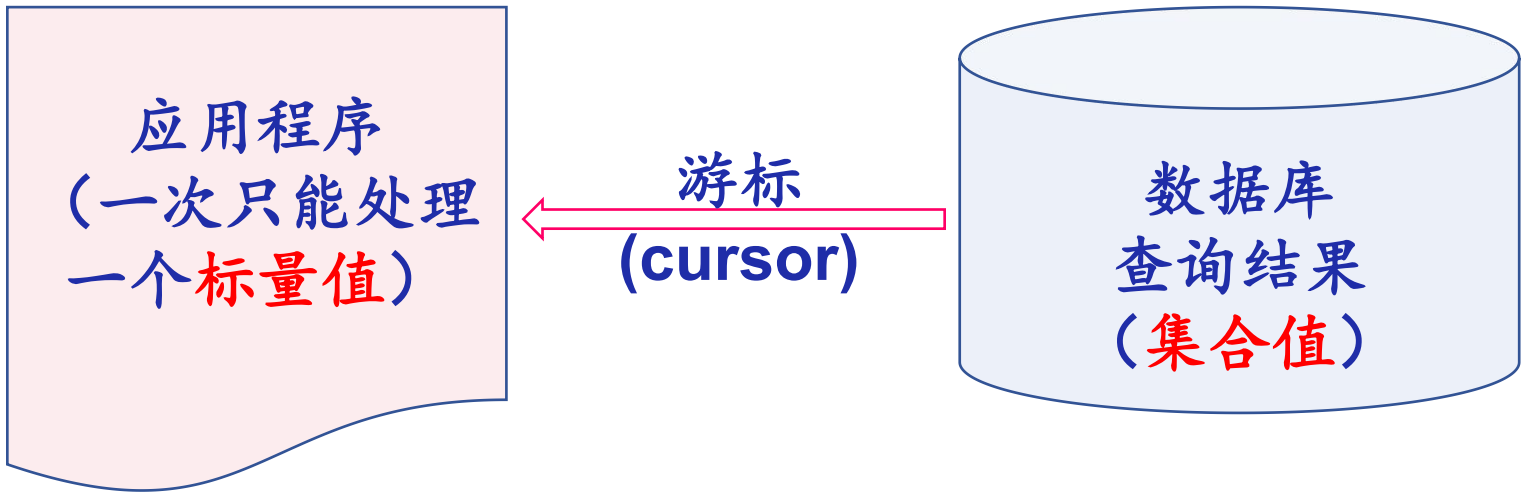
- 必须使用游标的SQL语句
  - 查询结果为多条记录的SELECT语句
  - CURRENT形式的UPDATE语句
  - CURRENT形式的DELETE语句

  } UPDATE/DELETE当前游标指向的元组

- 当一个查询只会返回单条结果元组，或者不确定该查询是否肯定返回单条结果元组时，也可以使用游标来完成SQL语句与主语言之间的数据交换。

1. 查询结果为多条记录的SELECT语句

- 在数据交换中，数据库SQL中的变量是集合型的而程序设计语言中的主变量则是标量型，因此数据库中SQL变量不能直接供程序设计语言使用，而需要有一种机制将SQL变量中的集合量逐个取出后送入应用程序主变量内供其使用，而提供此种机制就是游标（cursor）。



- 基于游标的数据交换特点：One-Row-at-a-Time Principle

1. 查询结果为多条记录的SELECT语句（续）

- 游标（Cursor）操作

(1) 声明游标 declare a cursor

- 为某一映像语句(可能返回多个结果元组)的结果集合定义一个命名的游标

(2) 打开游标 open the cursor

- 执行相应的映像语句并打开获得的结果集，此时游标处于活动状态并指向结果集合的第一条记录的前面

(3) 推进游标指针并取当前记录 fetch a row by the cursor

- 将游标推向结果集合中的下一条记录，读出游标所指向记录的值并赋给对应的主语言变量（One-Row-at-a-Time Principle）

(4) 关闭游标 close the cursor

- 关闭所使用的游标，释放相关的系统资源

（1）声明游标

```
EXEC SQL DECLARE cursor-name CURSOR FOR
        subquery
        [ ORDER BY ..... ]
        [ FOR { READ ONLY |
                UPDATE [ OF columnname, ..... ] } ] ;
```

- 是一条说明性语句，这时数据库管理系统并不执行SELECT语句；
- 如果一条查询语句的执行返回多条结果元组，那么必须使用游标来获取结果集合中的每一个结果元组；
- 仅当用户确信只可能返回单个结果元组的情况下才可以使用SELECT……INTO…… 形式的查询语句。

声明游标示例

define the cursor name

```
EXEC SQL DECLARE agent_dollars CURSOR FOR
    select  aid, sum(dollars)
    from    orders
    where  cid = :cust_id
    group by  aid ;
```

means multiple rows in result set

search by customer's id (stored in host variable cust_id) when open the cursor agent_dollars

（2）打开游标

- 打开游标实际上是执行相应的**SELECT**语句，把查询结果取到缓冲区中这时游标处于活动状态，指针指向查询结果集中的第一条记录

before open the cursor, you must place cno value of customer's id in the host variable cust_id using in the declare statement of cursor agent_dollars.

```
.....
EXEC SQL OPEN agent_dollars ;
.....
```

execute the select statement

after open the cursor, the pointer of the cursor has been placed in the position before the first row in result set.

（3）推进游标指针并取当前记录

- 推动游标指针至下一条记录，同时将缓冲区中当前游标指针指向的记录（即当前记录）取出来送至主变量供主语言进一步处理

```
while (TRUE) {                    /* loop to fetch rows */
    exec  sql  fetch agent_dollars
                  into :agent_id, :dollar_sum;

    printf("%s %11.2f\n", agent_id, dollar_sum);
}                                 /* end fetch loop */
```

① move the pointer of cursor to the next row, then the next row is current row

② fetch the current row's value into host variables: agent's id to agent_id, summation of dollars to dollar_sum

结束游标循环

- 通过whenever ... goto ...定义循环结束方式

declare ‘not found’ event processing

```
exec sql whenever not found goto finish;
   ......
while (TRUE) {
   exec sql fetch ..... into ......;
      ......
}
   ......
finish:   exec sql close agent_dollars;
```

execute this statement after fetch loop when ‘not found’ event is occur

(4) 关闭游标

- 关闭游标，释放结果集占用的缓冲区及其他资源

```
EXEC SQL CLOSE agent_dollars ;
```

① close the cursor, and release the result set and other resource in DBMS

② after close the cursor, it can be opened again

说明：

- 游标一旦被声明(declare)，可以被重复使用(open-fetch-close)。每一次open一个游标，都将重新执行对应的query，并生成新的结果集。
- 一个游标结果集只能被遍历一次，其中的结果元组被fetch顺序是随机的（游标定义中无order by子句）。
- 应用程序可以通过'游标状态变量'来了解一个游标的当前状态(是否处于打开状态、结果元组的个数、是否fetch到结果元组.....)

可滚动游标（Scrollable Cursors）

- 如果希望能够重复FETCH游标结果集中的元组，可使用可滚动游标

```
EXEC SQL DECLARE cursor_name [ INSENSITIVE ] [ SCROLL ]
    CURSOR [ WITH HOLD ] FOR
        subquery { UNION subquery }
        [ ORDER BY ..... ]
    [ FOR READ ONLY |
        FOR UPDATE OF columnname ..... ];
```

```
EXEC SQL FETCH
        [ { NEXT | PRIOR | FIRST | LAST |
          { ABSOLUTE | RELATIVE } value_spec } FROM ]
    cursor_name INTO .....;
```

2. CURRENT形式的UPDATE语句和DELETE语句

- 非CURRENT形式的UPDATE语句和DELETE语句，即常规的、不使用游标的UPDATE和DELETE操作，其特点是：
  - 面向集合的操作
  - 一次修改或删除所有满足条件的记录

- 在一个可更新游标上，可以使用CURRENT形式的UPDATE语句和DELETE语句，修改或删除当前游标记录
- [思考] 什么是可更新游标？

```
EXEC SQL DECLARE cursor-name CURSOR FOR
        subquery
        [ ORDER BY ..... ]
        [ FOR { READ ONLY |
                UPDATE [ OF columnname, ..... ] } ] ;
```

2. CURRENT形式的UPDATE语句和DELETE语句（续）

❑ CURRENT形式的UPDATE语句和DELETE语句的用途

- ➢ 如果只想修改或删除其中某个记录
  - 用带游标的SELECT语句查出所有满足条件的记录；
  - 从中进一步找出要修改或删除的记录；
  - 用CURRENT形式的UPDATE或DELETE语句修改或删除之
    - 在UPDATE语句或DELETE语句中要用子句
      WHERE CURRENT OF <游标名>
    - 表示修改或删除最近一次取出的记录，即游标指针当前指向的记录

❑ 不能使用CURRENT形式的UPDATE语句和DELETE语句

- ➢ 当游标定义中的SELECT语句带有UNION或ORDER BY子句
- ➢ 该SELECT语句相当于定义了一个不可更新的视图

例8.1：CURRENT形式的UPDATE语句示例

```
EXEC SQL DECLARE SX CURSOR FOR  /*定义游标SX*/
        SELECT Sno,Sname,Ssex,Sage  /*游标SX对应的查询语句*/
        FROM Student
        WHERE SDept = :deptname;
  ......
EXEC SQL OPEN SX;  /*打开游标SX，执行游标对应的查询，并指向查询结果的第一行*/
for ( ; ; )  /*用循环结构逐条处理结果集中的记录*/
{
    EXEC SQL FETCH SX INTO :HSno,:Hsname,:HSsex,:HSage;
            /*推进游标，将当前数据放入主变量*/
    if (SQLCA.SQLCODE != 0)  /* SQLCODE != 0，表示操作不成功 */
        break;  /* 利用SQLCA中的状态信息决定何时退出循环 */
    ......
    EXEC SQL UPDATE Student
        SET Sage = :NEWAGE
        WHERE CURRENT OF SX;  /* 嵌入式SQL更新语句 */
                              /*对当前游标指向的学生年龄进行更新*/
    ......
}
EXEC SQL CLOSE SX;  /*关闭游标SX，不再和查询结果对应*/
```

8.1.1 嵌入式SQL的处理过程

8.1.2 嵌入式SQL语句与主语言之间的通信

8.1.3 不用游标的SQL语句

8.1.4 使用游标的SQL语句

8.1.5 动态SQL

8.1.5 动态SQL

- **静态嵌入式SQL**
  - 静态嵌入式SQL语句能够满足一般要求
  - 但无法满足：到执行时才能够确定要提交的SQL语句、查询的条件

- **动态嵌入式SQL**
  - 在嵌入式SQL编程中，很多时候编程人员无法确定到底应该做什么工作，所使用的SQL语句也不能预先确定，需要根据程序的实际运行情况来决定，也就是根据实际情况来生成并调用SQL语句。这样的SQL语句被称为动态SQL。

- **动态SQL语句的可变性**
  - SQL语句正文动态可变
  - 变量个数动态可变
  - 语句类型动态可变
  - SQL语句引用对象动态可变

静态SQL & 动态SQL

	描述	优点	缺点
静态SQL	在应用开发阶段就能够确定下来的SQL语句。	在预编译时，DBMS将对SQL语句进行分析和执行计划优化，可确保数据库访问的正确性。	只能根据缺省参数值进行优化，其访问路径并非是最优路径。
动态SQL	在运行时动态生成的SQL语句。	可以根据运行时数据库的当前状态选择最优访问路径。	动态地进行SQL语句的语法分析和访问路径选择，需要额外的执行开销。

- 在什么情况下需要使用动态SQL?
  - 应用程序需要在执行过程中生成SQL语句；
  - SQL语句用到的数据库对象在预编译时不存在；
  - 希望SQL语句的执行能够根据执行时的数据库系统内部的统计信息来采用最优的访问策略。

8.1.5 动态SQL

1. 使用SQL语句主变量
2. 动态参数
3. 执行准备好的语句（EXECUTE）

1. 使用SQL语句主变量

- **SQL语句主变量**
  - 程序主变量包含的内容是SQL语句的内容，而不是原来保存数据的输入或输出变量
  - SQL语句主变量在程序执行期间可以设定不同的SQL语句，然后立即执行

- **[例8.6] 创建基本表TEST。**

```
EXEC SQL BEGIN DECLARE SECTION;
    const char *stmt = "CREATE TABLE test(a int);" ;
            /* SQL语句主变量，内容是创建表的SQL语句 */
EXEC SQL END DECLARE SECTION;
.....
EXEC SQL EXECUTE IMMEDIATE :stmt;   /*执行动态SQL语句*/
```

2. 动态参数

- 动态参数
  - SQL语句中的可变元素
  - 使用参数符号（?）表示该位置的数据在运行时设定
- 和主变量的区别
  - 动态参数的输入不是编译时完成绑定
  - 而是通过 PREPARE语句准备主变量和执行语句EXECUTE绑定数据或主变量来完成
- 使用动态参数的步骤

① 声明SQL语句主变量

② 准备SQL语句（PREPARE）

EXEC SQL PREPARE <语句名> FROM <SQL语句主变量>;

3. 执行准备好的语句（EXECUTE）

```
EXEC SQL EXECUTE <语句名>
    [INTO <主变量表>]
    [USING <主变量或常量>];
```

- [例8.7] 向TEST中插入元组。

```
EXEC SQL BEGIN DECLARE SECTION;
    const char *stmt = "INSERT INTO test VALUES(?);" ;
    /*声明SQL主变量内容是INSERT语句 */
EXEC SQL END DECLARE SECTION;
...
EXEC SQL PREPARE mystmt FROM :stmt;   /*准备语句*/
...
EXEC SQL EXECUTE mystmt USING 100;
          /*执行INSERT语句，设定参数？值为 100 */
EXEC SQL EXECUTE mystmt USING 200;
          /* 执行INSERT语句，设定参数？值为 200 */
```

8.1.1 嵌入式SQL的处理过程

8.1.2 嵌入式SQL语句与主语言之间的通信

8.1.3 不用游标的SQL语句

8.1.4 使用游标的SQL语句

8.1.5 动态SQL

8.1.6 嵌入式SQL总结

8.1.6 嵌入式SQL（总结）

1. 主语言语句 与 SQL语句 的区别？
   - 对嵌入在主语言中的SQL语句必须加前缀和后缀
   - 当主语言是C语言时，前缀是EXEC SQL，后缀是END_EXEC 或 分号 ‘;’

2. 主语言程序与ESQL间的通讯
   - 向主语言传递SQL语句的执行状态：SQL通讯区 -- SQLCA
   - 主语言向SQL语句提供参数：主变量 (host variable)
   - 将SQL语句的查询结果交换给主语言处理：游标 (cursor) + 主变量

3. 主变量 与 SQL变量 的区别？
   - 可以在嵌入式SQL语句中使用的主语言变量被称为‘主变量’；SQL语句中的表名或属性名被称为‘SQL变量’，SQL变量只能在嵌入式SQL语句中使用。
   - 在嵌入式SQL语句中，为了区分‘主变量’和‘SQL变量’，需要在‘主变量’标识符的前面加上一个前缀‘:’，以便与表名或属性名等‘SQL变量’区别开来。
   - 可以通过‘主变量’在嵌入式SQL语句与主语言语句之间交换数据：可以通过主变量获取查询结果值，并用于主语言语句中；也可以将保存在主语言变量中的值用于SQL语句的执行。

4. SQL查询语句 与 主语言语句 的结果数据交换？
   - 单条结果元组的数据交换：SELECT ... INTO ... 或 游标
   - 多条结果元组的数据交换：游标

Some Common ESQL Statement

❑ Type Correspondences

Basic SQL type	ORACLE type	DB2 UDB type	C datatype
char(n)	char(n)	char(n)	char arr[n+1]
varchar(n)	varchar(n)	varchar(n)	char array[n+1]
smallint	smallint	smallint	short int
integer, int	integer, int, number(10)	integer, int	int
real	real	real	float
double precision, float	double precision, number, float	double precision, double, float	double

Some Common ESQL Statement

❑ Basic Embedded SQL Select Form (Single-Row Select)

```
EXEC SQL
    SELECT [ ALL|DISTINCT ] expression, ......
    INTO  host-variable [ indicator-variable ], ......
    FROM  tableref [corr-name], ......
    [ WHERE  search-condition ]  ;
```

❑ Embedded SQL Declare Cursor Syntax

```
EXEC SQL
    DECLARE cursor_name CURSOR FOR
    subquery
    [ ORDER BY ..... ]
    [ FOR { READ ONLY |
                UPDATE [ OF columnname, ..... ] } ]  ;
```

Some Common ESQL Statement

Embedded Basic SQL Delete Syntax

```
EXEC SQL
    DELETE FROM tablename [ corr_name ]
    [ WHERE search_condition |
      WHERE CURRENT OF cursor_name ]
```

Embedded Basic SQL Update Syntax

```
EXEC SQL
    UPDATE tablename [ corr_name ]
    SET columnname = expr, ......
    [ WHERE search_condition |
      WHERE CURRENT OF cursor_name ]
```

Some Common ESQL Statement

❑ Embedded Basic SQL Insert Syntax

```
EXEC SQL
    INSERT INTO tablename [ (column_nme, ... ) ]
    VALUES ( expr, ..... )  |  subquery   ;
```

❑ The Other ESQL Statement

```
EXEC SQL CREATE TABLE ..... ;
EXEC SQL DROP TABLE ..... ;
EXEC SQL COMMIT WORK ;
EXEC SQL ROLLBACK WORK ;
EXEC SQL CONNECT ..... ;
EXEC SQL DISCONNECT ..... ;
```

第8章　数据库编程

8.2 过程化SQL

(2026 春季)

第8章 数据库编程

8.1 嵌入式SQL

8.2 过程化SQL

8.3 存储过程和函数

8.4 ODBC编程

8.5* OLE DB

8.6* JDBC编程

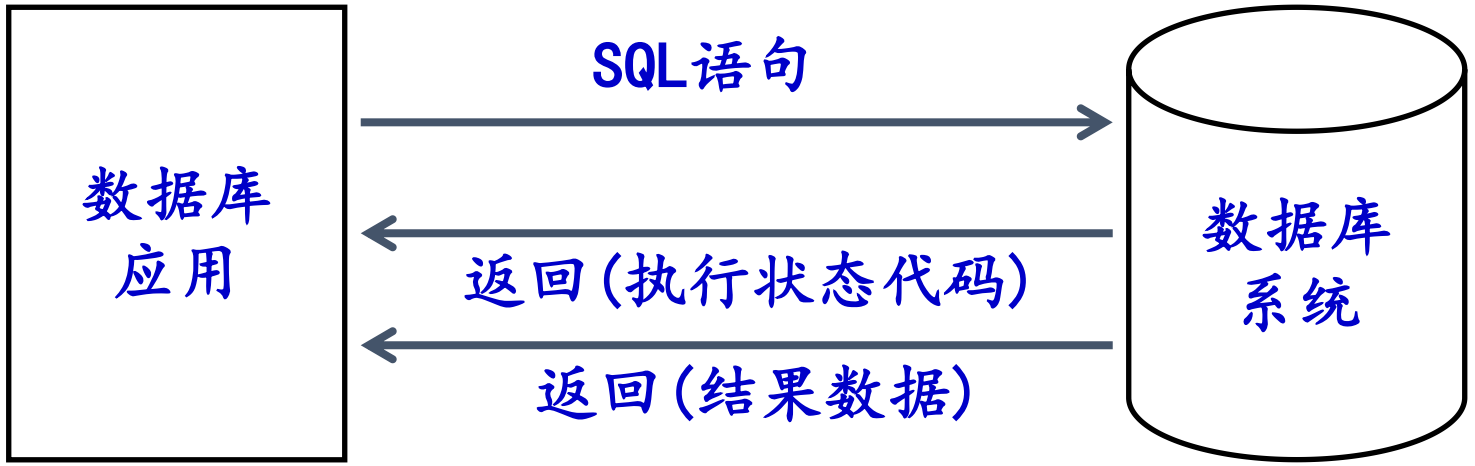
8.7 小结

review：学习导引

❑ SQL语言的三种使用方式

	使用方式	应用场景
交互式SQL	命令行 / 批处理	可独立运行，一般供临时用户操作访问数据库用(即席查询, ad-hoc query)
嵌入式SQL	主语言 + ESQL	数据库应用开发
过程化SQL	SQL编程	• 兼有SQL数据访问和高级程序设计语言的流程控制、简单数值处理功能； • 可在数据库服务器中独立运行； • 常用于编写存储过程、存储函数、触发器。

❑ 数据交换模型



8.2 过程化SQL

- 为了能够在数据库中实现部分应用程序逻辑，需要在交互式SQL语言的基础上扩充部分过程式程序设计语言的成份，构成一个可编程的SQL语言，称为过程化SQL，又被称为过程式SQL、可编程SQL。
- 过程化SQL的作用：
  - 可用于定义触发器、存储过程、存储函数对应的SQL程序块
- 过程化SQL相对于嵌入式SQL的优势
  - 可独立编程，不再需要区分 主变量 与 SQL变量
  - 不需要经历从预编译到编译的处理
  - 在数据库服务器内部实现数据交换与处理
- 最常用的两种过程化SQL： PL/SQL and T-SQL

8.2 过程化SQL - 学习导引

- 过程化SQL的内容
  - 交互式SQL语句
  - SQL中的数据交换，包括游标、诊断及动态SQL
  - 事务控制
  - 传统程序设计语言的主要成份，包括：
    - 变量、常量、参数的定义
    - 流程控制及循环语句：
      - begin.....end块，if...then...else结构，while/for循环结构，case...switch结构，异常处理块
      - return语句
    - 函数/过程调用语句
    - 服务性的函数库、类库等

8.2.1 过程化SQL的块结构

8.2.2 变量和常量的定义

8.2.3 流程控制

8.2.1 过程化SQL的块结构

- 过程化SQL
  - 是对SQL的扩展，增加了过程化语句功能
  - 过程化SQL的基本结构是‘块’（图8.2）
    - 块之间可以互相嵌套
    - 每个块完成一个逻辑操作

过程化SQL块

定义部分

```
DECLARE        /* 定义的变量、常量等只能在该基本块中使用 */
变量、常量、游标、异常等 /* 当基本块执行结束时，定义就不再存在 */
```

执行部分

```
BEGIN
    SQL语句、过程化SQL的流程控制语句
EXCEPTION
    异常处理部分
END;
```

/* 遇到不能继续执行的情况称为‘异常’*/

/* 在出现异常时，采取措施来纠正错误或报错 */

8.2.1 过程化SQL的块结构

8.2.2 变量和常量的定义

8.2.3 流程控制

8.2.2 变量和常量的定义

1. 变量定义

变量名 数据类型 [[NOT NULL] := 初值表达式]

变量名 数据类型 [[NOT NULL] 初值表达式]

2. 常量定义

常量名 数据类型 CONSTANT := 常量表达式

- 常量必须要给一个值，并且该值在存在期间或常量的作用域内不能改变。如果试图修改它，过程化SQL将返回一个异常。

3. 赋值语句

- 变量名称 := 表达式

PL/SQL中的变量定义

Oracle 中常用的基本数据类型

数据类型	描述
Number	数字型，范围10^{-130}~10^{125}
Char	字符型，最大2000个字符
Date	日期型，包括日期、小时、分、秒
Long	大文本
Boolean	逻辑型，取值true,false,null
Varchar2	变长字符型，最大为4000个字符

PL/SQL中的变量定义（续）

- 简单变量定义的一般格式：

<*variablename*> [CONSTANT]<*datatype*> [[NOT NULL] {DEFAULT|:=} <*expression*>];

- 说明
  - 在**PL/SQL**中使用的变量、常量、游标和异常处理的名字都必须先定义后使用。
  - 定义部分是包括在保留字**DECLARE**和**BEGIN**之间的部分，每条语句后用‘;’结束。
  - 每行只能定义一个标识符。
  - 如果加上保留字**CONSTANT**，则表示所定义的标识符为一个常量，必须为它赋初值。
  - 标识符的命名规则与**SQL**的规则基本相同，即每个标识符必须以字母开头，而且不分大小写。
  - 为标识符赋值时，使用赋值符号‘**:=**’，默认值为空。

PL/SQL中的变量定义（续）

- 变量定义的几种方式

1. 基本数据类型
2. %TYPE 类型
3. %RECORD 类型
   ① 自定义记录类型
   ② 基于数据库表结构的记录类型

PL/SQL的变量定义示例（1）

1、使用基本数据类型定义变量

```
sex  BOOLEAN :=TRUE;

today  DATE not null:=sysdate;

age  number(3) not null:=25;
```

PL/SQL的变量定义示例（2）

2、%TYPE类型

- 声明一个变量，使它的类型与某个变量或数据库表中某个列的数据类型一致
- 当我们不知道该变量或列的数据类型时，可以使用 **%TYPE**

```
DECLARE
teacher_name char（5）;
student_name teacher_name%TYPE;
BEGIN
……
END
```

```
DECLARE
no EMP.EMPNO%TYPE;
BEGIN
……
END
```

PL/SQL的变量定义示例（3）

3、记录类型定义的一般格式（自定义记录类型）

```
TYPE <recordtypename> IS RECORD (
    <field> <datatype> [[NOT NULL]{DEFAULT|:=} <expression>]
    [,<field> …]
);
```

- 说明：
  - 标识符 <recordtypename>是定义的记录类型名；
  - 要定义记录型变量，定义方法与前面标量型变量定义一样。
  - 记录类型变量的属性引用方法是‘.’引用。

PL/SQL的变量定义示例（4）

- 声明记录类型和记录类型变量

```
DECLARE
    TYPE student IS RECORD （
        id NUMBER(4) NOT NULL default 0,
                        /* 非空时必须加上缺省值 */
        name CHAR(10) ,
        sex BOOLEAN NOT NULL default true,
        birthdate DATE,
        physics NUMBER(3),
        chemistry NUMBER(3)
    );
    /*下面定义一个student类型的变量 */
    student1  student;
```

PL/SQL的变量定义示例（5）

4、声明一个记录型变量，使它的类型与某个基本表的数据结构一致，可以使用%ROWTYPE的形式定义。

```
DECLARE
    emp_val  emp%ROWTYPE;
/* 定义一个与关系表emp的表结构相同的记录型变量emp_val */
```

- 变量引用方式
  - emp_val.empno　　/* empno是表emp中的一个属性 */
  - emp_val.ename　　/* ename也是表emp中的一个属性 */

PL/SQL中的全局变量与局部变量

- 局部变量
  - 在过程块中定义的变量
  - 只能在块内引用
- 全局变量
  - 在过程块之外定义的变量
  - 使用时需要加':'前缀

```
variable fac number;  /*全局变量*/
DECLARE
    NUM NUMBER(3):=5;  /*局部变量*/
BEGIN
    :fac := 1;
    if NUM>0 then
        for  i  in  1..NUM  loop
            :fac := :fac * i;
        end loop;
    end if;
END;
```

PL/SQL中的游标属性

- 游标名%ISOPEN
  - 布尔型。如果游标已经打开，返回TRUE，否则返回FALSE。
- 游标名%FOUND
  - 布尔型。如果最近一次使用FETCH语句，有返回结果则为TRUE，否则为FALSE;
- 游标名%NOTFOUND
  - 布尔型。如果最近一次使用FETCH语句，没有返回结果则为TRUE，否则为FALSE;
- 游标名%ROWCOUNT
  - 数值型。描述的是到目前为止实际从游标工作区取的记录数。

8.2.1 过程化SQL的块结构

8.2.2 变量和常量的定义

8.2.3 流程控制

8.2.3 流程控制

- 过程化SQL功能
  1. 条件控制语句
  2. 循环控制语句
  3. 错误处理

PL/SQL中的流程控制 - 条件控制语句

1. 条件控制语句：IF-THEN，IF-THEN-ELSE 和 嵌套的IF语句

（1）

```
IF condition THEN
    Sequence_of_statements;
END IF;
```

（2）

```
IF condition THEN
    Sequence_of_statements1;
ELSE
    Sequence_of_statements2;
END IF;
```

（3）在THEN和ELSE子句中还可以再包含IF语句，即IF语句可以嵌套

2. 循环控制语句：LOOP，WHILE-LOOP和FOR-LOOP

(1) 简单的循环语句LOOP

```
LOOP
        Sequence_of_statements;
END LOOP;
```

- 多数数据库服务器的过程化SQL都提供EXIT、BREAK 或 LEAVE等循环结束语句，保证LOOP语句块能够结束循环。

(2) WHILE-LOOP 循环

```
WHILE  condition  LOOP
        Sequence_of_statements;
END LOOP;
```

- 每次执行循环体语句之前，首先对条件进行求值。如果条件为真，则执行循环体内的语句序列；如果条件为假，则跳过循环并把控制传递给下一条语句。

(3) FOR-LOOP 循环

```
FOR  count  IN [REVERSE]  bound1 … bound2  LOOP
        Sequence_of_statements;
END LOOP;
```

3. 错误处理

- 如果过程化SQL在执行时出现异常，则应该让程序在产生异常的语句处停下来，根据异常的类型去执行异常处理语句。
- SQL标准对数据库服务器提供什么样的异常处理做出了建议，要求过程化SQL管理器提供完善的异常处理机制。

❑ 在PL/SQL中，异常处理的一般格式

```
EXCEPTION
    WHEN 异常情况1 [OR 异常情况2…] THEN
        …… ;
    WHEN异常情况3 [OR 异常情况4…] THEN
        …… ;
    WHEN OTHERS THEN
        …… ;
```

T-SQL的基本语法

标识符

- **标识符的含义**
  - 是指用户定义的变量名、函数名、存储过程名、库名、表名、索引名、视图名等各种名称。

- **命名规则**
  - 标识符的长度可以为1-127个字符
  - 标识符的第一个字符必须为字母、下划线、@、#
    - 以@为首的标识符表示局部变量
    - 以@@为首的标识符表示全局变量
    - 以#为首的标识符表示1个临时数据库对象
    - 对于表或存储过程，名称前包含一个#时表示局部临时对象，而两个#(“##”)则表示为全局临时对象。
  - 标识符内不允许有空格，也不允许为SQL中的保留字

变量

- 局部变量
  - 用户自己定义的变量，用于在语句之间传递数据
  - 局部变量名以@符号开头
  - 使用时，必须先声明

- 全局变量
  - 是系统定义的变量，用于存储系统的特定信息
  - 用户不能建立全局变量，也不能修改其值
  - 全局变量名以@@符号开头

几个比较重要的全局变量

@@ERROR

返回最后执行的 Transact-SQL 语句的错误代码。

@@ROWCOUNT

返回受上一语句影响的行数。

@@CURSOR_ROWS

返回连接上最后打开的游标中当前存在的合格行的数量。

@@FETCH_STATUS

返回被 FETCH 语句执行的最后游标的状态，

局部变量的声明、赋值、显示

- 局部变量的声明

  declare @变量名 数据类型 [, …]

- 局部变量的赋值（两种方法）

  ① set @variable_name = expression

  ② select @variable_name = expression …

- 局部变量的显示（两种方式）

  ① print expression

  ② select expression [, …]

  - print只能用于char、varchar类型值的显示；或可以隐式地转化成字符型的常量或变量表达式。

局部变量的赋值

- 使用SELECT语句为局部变量赋值

```
USE Northwind
GO
DECLARE @EmpIDVariable INT
SELECT @EmpIDVariable = EmployeeID
FROM Employees
ORDER BY EmployeeID DESC
SELECT @EmpIDVariable
GO
```

常见函数

- 字符串函数
  - 求串长len( )、转换为大写/小写字母upper( ) / lower( )
- 数学函数
  - Sum( )、avg( )、max( )、min( )、count( )
- 日期和时间函数
  - 求当前系统日期：Getdate( )
- 数据类型转换函数
  - 转换为字符型数据：Convert( )、cast( )

常见运算符和优先级

优先级	运算符
1	+（正）、 -（负）、~（按位取反）
2	*（乘）、/（除）、%（取模）
3	+（加）、+（字符串连接）、 -（减）
4	=、 >、 <、 >=、 <=、 <>、 !=、 !>、 !<
5	&（位与）、 |（位或）、^（位异或）
6	NOT
7	AND
8	ALL、ANY、BETWEEN、IN、LIKE、OR、SOME
9	=（赋值）

T-SQL的流程控制语句

- **Transact-SQL**命令除了可以单独使用以外，还可以使用流程控制语句把多个命令组合成一段程序，用以完成各种更为复杂的程序算法。

  - BEGIN ..... END
  - IF ..... ELSE .....
  - CASE
  - WHILE, CONTINUE, BREAK
  - GOTO
  - WAITFOR
  - RETURN

例: 搜索型CASE表达式的使用

```
USE pubs
GO
SELECT    'Price Category' =
     CASE
       WHEN price IS NULL THEN 'Not yet priced'
       WHEN price < 10 THEN 'Very Reasonable Title'
       WHEN price >= 10 and price < 20 THEN 'Coffee Table Title'
       ELSE 'Expensive book!'
     END,
   CAST(title AS varchar(20)) AS 'Shortened Title'
FROM titles
ORDER BY price
GO
```

例：WHILE、BREAK、CONTINUE的配合使用（1）

- 在该例子中，程序首先计算pubs数据库中所有书籍的平均价格，如果平均价格低于30美圆，则所有书籍的价格都变为原来的两倍。然后选择最高价格。如果最高价格小于或者等于50美圆，那么重新开始循环并且将书价再次加倍。该循环一直翻倍价格直到最高价格超过50美圆，然后退出WHILE循环，并且打印一条警告信息。

例：WHILE、BREAK、CONTINUE的配合使用（2）

```
USE pubs
GO
WHILE ( SELECT AVG ( price ) FROM titles ) < $30
BEGIN
    UPDATE titles
    SET  price = price * 2
    SELECT MAX ( price ) FROM titles
    IF ( SELECT MAX ( price ) FROM titles ) > $50
            BREAK
    ELSE
            CONTINUE
END
PRINT  ‘Too much for the market to bear’
```

使用WAIT TIME语句

```
BEGIN
   WAITFOR TIME '22:20'
   EXECUTE update_all_stats
END
```

第8章　数据库编程

8.3 存储过程和函数

(2026 春季)

8.3 存储过程和函数

❑ 存储过程

- 用过程化SQL编写的过程，经编译和优化后存储在数据库服务器中，因而被称为‘存储过程’
- 可以被应用程序或其他存储过程调用

❑ 函数

- 函数的定义与存储过程类似，也是用过程化SQL编写，经编译和优化后存储在数据库服务器中，因而也被称为‘存储函数’或‘自定义函数’。
- 与存储过程不同之处是函数必须定义返回值的类型。

存储过程与函数的区别

- 使用场景不同
  - 存储过程通常用于复杂业务逻辑的实现和完成数据库运维管理任务
  - 函数一般用于计算和数据查询任务
- 结果返回方式不同
  - 存储过程本身没有返回值，只能通过访问存储过程的输出参数或输入/输出参数来获取执行结果；
  - 函数有返回类型定义和返回值，它把执行结果返回给调用者；
  - 因此，存储过程中可以没有返回语句RETURN，或者使用不带参数的返回语句；函数必须通过带参数的返回语句结束。
- 执行方式不同
  - 存储过程只能被直接调用执行
  - 函数可以出现在SQL语句和表达式中

使用存储过程/函数 的优点

- 存储过程/函数 的优点
  - 运行效率高
    - 已经经过编译和优化，在调用时不需要再进行语法分析和优化工作，运行效率高；
  - 降低客户机和数据库服务器之间的通讯开销
    - 只需要提交过程名和参数值，并返回最终的处理结果
  - 方便用户业务逻辑的实现和管理
    - 由数据库管理系统统一集中管理，既有利于集中管理，又便于维护

❑过程化SQL块类型

❑ 命名块

- 编译后保存在数据库中，可以被反复调用，运行速度较快。
- 过程 和 函数 是命名块

❑ 匿名块

- 每次执行时都要进行编译，它不能被存储到数据库中，也不能在其他过程化SQL块中被调用。

```
DECLARE
    ......  /* 定义声明部分 */
BEGIN                                  }
    ......                             }
    EXCEPTION                          }  /* 执行部分 */
    ...... /* 异常处理部分部分 */      }
END                                    }
```

8.3.1 存储过程

8.3.2 函数

8.3.3 过程化SQL中的游标

8.3.1 存储过程

- 存储过程
  - 由过程化SQL语句书写的过程，经编译和优化后存储在数据库服务器中，使用时只要调用即可。

- 存储过程的优点
  - 运行效率高
    - 在创建时进行语法分析和优化工作，提高了运行效率
  - 降低了客户机和服务器之间的通信量
  - 方便实施企业规则
    - 由数据库管理系统统一集中管理，既有利于集中管理，又便于维护

8.3.1 存储过程

- 存储过程的用户接口
  - （1）创建存储过程
  - （2）执行存储过程
  - （3）修改存储过程
  - （4）删除存储过程

存储过程的用户接口-创建存储过程 1

❑ 创建存储过程

CREATE [OR REPLACE] PROCEDURE 过程名([参数1,参数2,...]) /* 存储过程首部 */

AS <过程化SQL块>; /*存储过程体，描述该存储过程的操作 */

- 过程名：存储过程的过程名
- 参数列表：
  - 用参数名来标识调用时给出的参数值，必须指定参数值的数据类型。
  - 参数也可以被定义为 输入参数、输出参数 或 输入/输出参数
    - 参数名 **IN ...** /* 输入参数，也是默认的参数类型 */
    - 参数名 **OUT ...** /* 输出参数，用于存放存储过程的执行结果 */
    - 参数名 **IN OUT ...** /* 输入/输出参数 */
- 过程体：是一个<过程化SQL块>，包括声明部分和可执行部分（包括可执行的过程化SQL语句，以及可能的异常处理部分）

存储过程的用户接口-创建存储过程 2

- [例8.8] 利用存储过程来实现下面的应用：从账户1转指定数额的款项到账户2中。

```
CREATE OR REPLACE PROCEDURE TRANSFER
    ( inAccount  INT,  outAccount  INT,  amount  FLOAT )
    /* 定义存储过程TRANSFER，如果该过程已经存在则覆盖其原有定义 */
    /* 三个输入参数：转入账户、转出账户、转账额度 */
AS
DECLARE              /* 定义变量 */
    totalDepositOut  Float;
    totalDepositIn  Float;
    inAccountnum  INT;
```

存储过程的用户接口-创建存储过程 3

```
BEGIN    /* 过程体执行部分的开始 */
    /* 检查转出账户及其余额 */
    SELECT Total INTO totalDepositOut FROM Accout WHERE accountnum=outAccount;

    IF  totalDepositOut  IS NULL  THEN   /* 转出账户不存在或账户中没有存款 */
        ROLLBACK;              /* 回滚事务 */
        RETURN;
    END IF;

    IF  totalDepositOut < amount  THEN        /*如果账户存款不足*/
        ROLLBACK;         /* 回滚事务 */
        RETURN;
    END IF;
```

存储过程的用户接口-创建存储过程 4

```
    /* 检查转入账户 */
    SELECT Accountnum INTO inAccountnum FROM Account WHERE accountnum=inAccount;

    IF  inAccount  IS NULL  THEN        /* 如果转入账户不存在 */
        ROLLBACK;     /* 回滚事务 */
        RETURN;
    ENDIF;

    /* 修改转出账户余额，减去转出额 */
    UPDATE  Account  SET total = total - amount  WHERE  accountnum = outAccount;
    /* 修改转入账户余额，增加转入额 */
    UPDATE  Account  SET  total = total + amount  WHERE  accountnum = inAccount;

    COMMIT;     /* 提交转账事务 */
END;    /* 过程体执行部分的结束 */
```

存储过程的用户接口-执行存储过程

- 执行存储过程

  CALL/PERFORM PROCEDURE 过程名([参数1,参数2,...]);

  - 使用CALL或者PERFORM等方式激活存储过程的执行
  - 在过程化SQL中，数据库服务器支持在过程体中调用其他存储过程

- [例8.9] 从账户01003815868转10000元到01003813828账户中。

```
CALL PROCEDURE TRANSFER(01003813828,01003815868,10000);
```

存储过程的用户接口-其他

- 修改存储过程：对存储过程重命名或重编译
  - 重命名

    ALTER PROCEDURE 过程名1 RENAME TO 过程名2;
  - 重新编译

    ALTER PROCEDURE 过程名 COMPILE;

- 删除存储过程

  DROP PROCEDURE 过程名;

8.3 存储过程和函数

8.3.1 存储过程

8.3.2 函数

8.3.3 过程化SQL中的游标

8.3.2 函数

- 使用过程化SQL编写的‘函数’，也被称为‘存储函数’或‘自定义函数’

- 函数和存储过程的异同
  - 同：都是持久性存储模块
  - 异：函数必须指定返回的类型

❑ PL/SQL中的函数定义语句格式

```
CREATE [OR REPLACE] FUNCTION  function_name
    [(parameter1 [{IN|OUT|IN OUT}] datatype [{:=|DEFAULT} expression]
    [, parameter2 [{IN|OUT|IN OUT}] datatype [{:=|DEFAULT} expression]
    …])]
RETURN  returntype    /* 定义函数的返回值类型 */
{IS|AS}
    [declarations]
BEGIN
        code
    [EXCEPTION
        exception_handlers]
END
```

8.3.2 函数

- 函数的执行语句格式

  CALL/SELECT 函数名 ([参数1,参数2,…]);

- 修改函数
  - 重命名
    - ALTER FUNCTION 过程名1 RENAME TO 过程名2;
  - 重新编译
    - ALTER FUNCTION 过程名 COMPILE;

存储过程示例

```
/* 定义存储过程：连续输出某个字符 */
CREATE OR REPLACE PROCEDURE printLine
    (width IN INTEGER, chr IN CHAR DEFAULT '-')  IS
BEGIN
    FOR i IN 1..width LOOP
        dbms_output.put(chr);
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('');
END printLine;

/* 调用存储过程 */
begin
    printline(40, '*');
end;
```

函数示例

```
/* 定义函数：返回指定职工的总收入 */
CREATE OR REPLACE FUNCTION totalsal(v_empno in emp.empno%type)
     RETURN number   /* 定义函数返回值的数据类型 */
IS
    totalsal1 number;
    begin
        select  sal + comm  into  totalsal1
        from  emp
        where  empno = v_empno;
        return totalsal1;
    end;

/* 在SQL语句中调用函数 */
select empno, totalsal(empno) from emp where totalsal(empno)>300;
```

8.3.1 存储过程

8.3.2 函数

8.3.3 过程化SQL中的游标

8.3.3 过程化SQL中的游标

- 过程化SQL中的游标，与嵌入式SQL中的游标及其使用方式一样，只是在语句格式上稍有差别。

- [例8.11] 带参数的游标

```
CREATE OR REPLACE PROCEDURE proc_cursor() AS
DECLARE
    cno  CHAR(3);
    cname  CHAR(8);

    CURSOR  mycursor(leaderno  CHAR(3))  FOR  /* 带参数游标mycursor */
        SELECT lno, lname FROM leader WHERE lno=leaderno;
        /* 检索leader表中属性lno的值 等于 游标参数leaderno值的记录 */
```

[例8.11] 带参数的游标（续）

```
BEGIN
    OPEN  mycursor( 'L01' );   /* 使用参数 L01 打开游标 */
    FETCH  mycursor  INTO  cno, cname;   /* 获取lno='L01'的游标元组 */
    INSERT INTO temp(lno,lname) VALUES(cno,cname);
          /* 将游标元组插入临时表temp中 */
    CLOSE  mycursor;  /* 关闭游标 */

    OPEN  mycursor( 'L02' );   /* 使用新的参数 L02 重新打开游标 */
    FETCH  mycursor  INTO  cno, cname;   /* 获取lno='L02'的新的游标元组 */
    INSERT INTO temp(lno,lname) VALUES(cno,cname);
          /* 将新的游标元组插入临时表temp中 */
    CLOSE  mycursor;   /* 关闭游标 */
END;
```

用过程化SQL创建触发器（1）

- [例5.21]当对表SC的Grade列进行修改时，若分数增加了10%则将此次操作记录到表SC_U(Sno,Cno,Oldgrade,Newgrade)中去。其中：Oldgrade是修改前的分数，Newgrade是修改后的分数。（P170页）

```
CREATE TRIGGER  SC_T
AFTER UPDATE OF Grade ON SC
REFERENCING
                    OLD  row   AS  OldTuple,
                    NEW  row  AS  NewTuple
FOR EACH ROW
WHEN (NewTuple.Grade >= 1.1*OldTuple.Grade)
INSERT INTO SC_U(Sno,Cno,OldGrade,NewGrade)
VALUES(OldTuple.Sno, OldTuple.Cno, OldTuple.Grade, NewTuple.Grade);
```

用过程化SQL创建触发器（2）

- [例5.23] 定义一个BEFORE行级触发器，为教师表Teacher定义完整性规则“教授的工资不得低于4000元，如果低于4000元，自动改为4000元”。（P171页）

```
CREATE TRIGGER Insert_Or_Update_Sal
BEFORE INSERT OR UPDATE ON Teacher
    /*触发事件是插入或更新操作*/
REFERENCING NEW ROW AS newTuple
FOR EACH ROW       /*行级触发器*/
BEGIN    /*定义触发动作体，是一个PL/SQL过程块*/
    IF (newTuple.Job = '教授') AND (newTuple.Sal < 4000)
    THEN  newTuple.Sal := 4000;
    END IF;
END;
```

用过程化SQL创建触发器（3）

```
CREATE OR REPLACE TRIGGER logemp
BEFORE  INSERT OR UPDATE OR DELETE ON EMP
FOR  EACH  ROW
DECLARE
    statementtype CHAR(20);
BEGIN
    IF INSERTING THEN
        statementtype:='INSERT TRIGGER!';
    ELSIF UPDATING THEN
        statementtype:='UPDATE TRIGGER!';
    ELSE
        statementtype:='DELETE TRIGGER!';
    END IF;
    DBMS_OUTPUT.PUT_LINE(statementtype);
END;
```

小结

- 过程化SQL
- 存储过程 & 函数
- 如何用过程化SQL编写存储过程和函数

第8章　数据库编程

（基于驱动的数据库编程）

（2026 春季）

第8章 数据库编程

8.1 嵌入式SQL

8.2 过程化SQL

8.3 存储过程和函数

8.4 ODBC编程

8.5* OLE DB

8.6* JDBC编程

8.7 小结

面向驱动的数据库编程

- JAVA世界：JDBC
  - JDBC 简介_w3cschool
- 微软世界：ODBC，OLE DB等
- Python世界：
  - Python DB-API + 各数据库模块（如PyMySQL，SQLite3等）

面向驱动的数据库编程

- C/S方式下的数据库访问接口
  - ISO接口：SQL/CLI
  - 微软接口：ODBC
  - SUN接口：JDBC

- 数据交换的流程
  - 连接阶段
  - 数据交换阶段
  - 断开连接阶段

面向ODBC驱动的数据交换流程

阶段	任务	ODBC主要函数
连接阶段	负责资源分配和建立连接	AlloEnv 分配SQL环境 AlloStmt 分配SQL语句 AlloHandle 分配SQL资源 AlloConnect 分配SQL连接 Connect 创建连接
数据交换阶段	数据访问与交换	有关游标的函数 有关诊断的函数 有关动态SQL的函数 有关数据获取的函数 其它函数
断开连接阶段	断开连接并释放资源	SQLFreeStmt 释放语句句柄 SQLDisConnect 撤销与数据库的连接 SQLFreeConnect 释放数据库连接句柄 SQLFreeEnv 释放环境句柄

图8.3 ODBC应用系统的体系架构

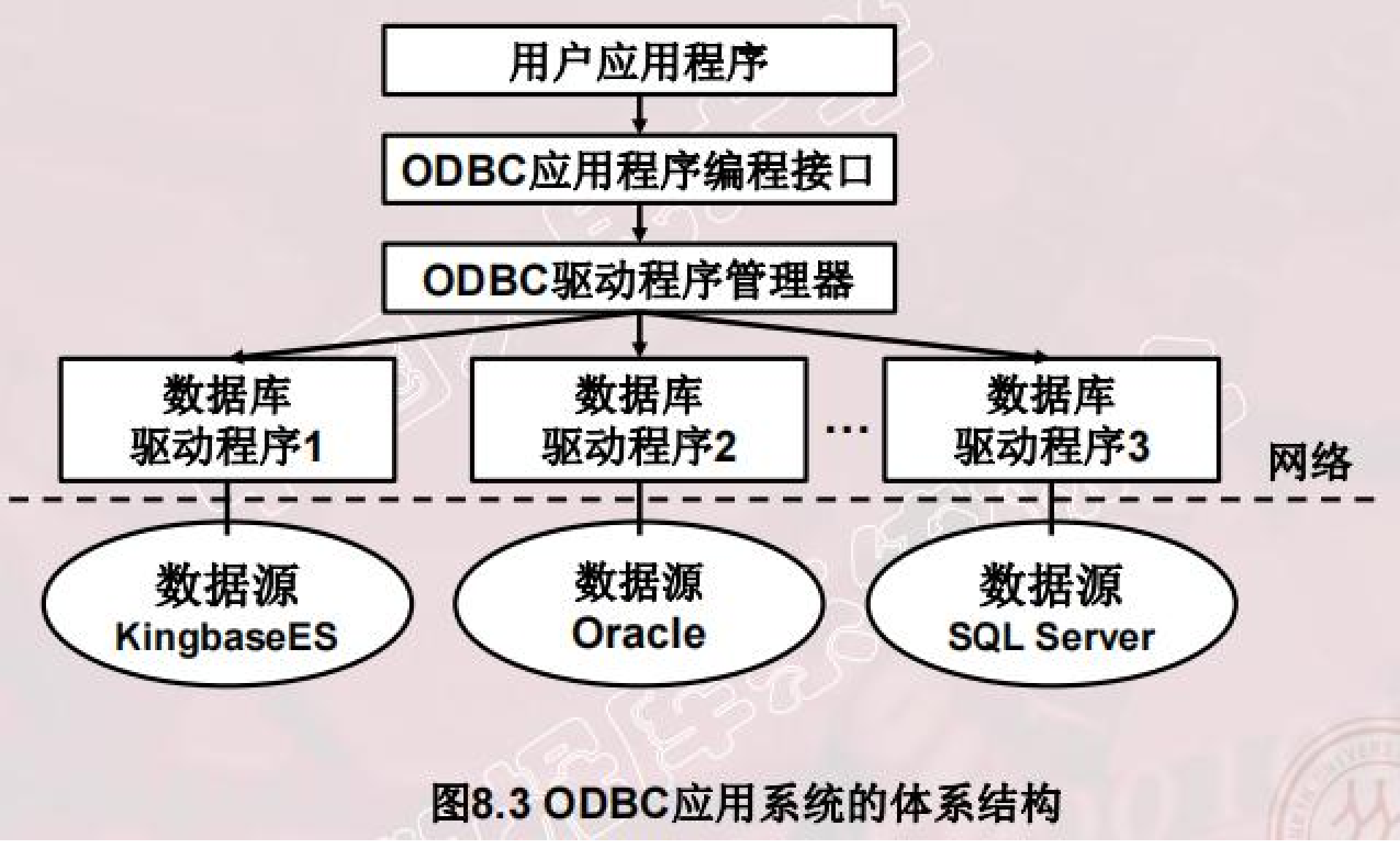
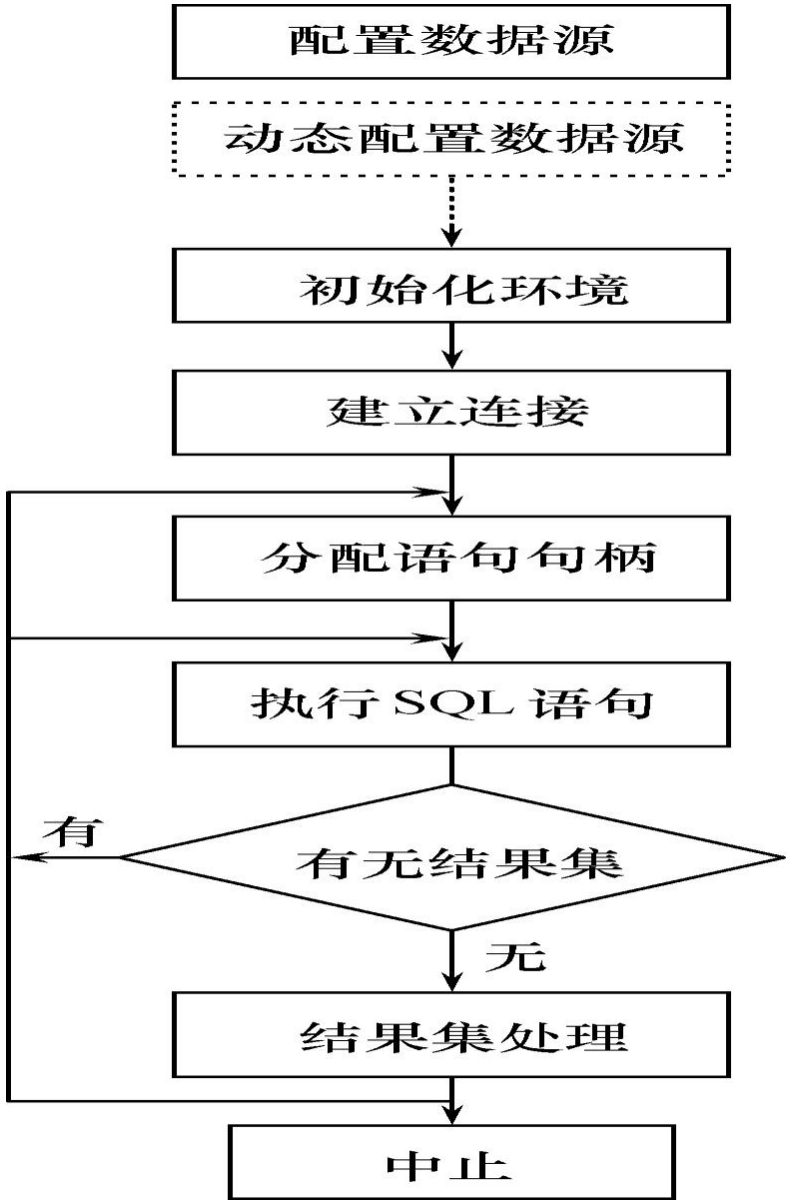
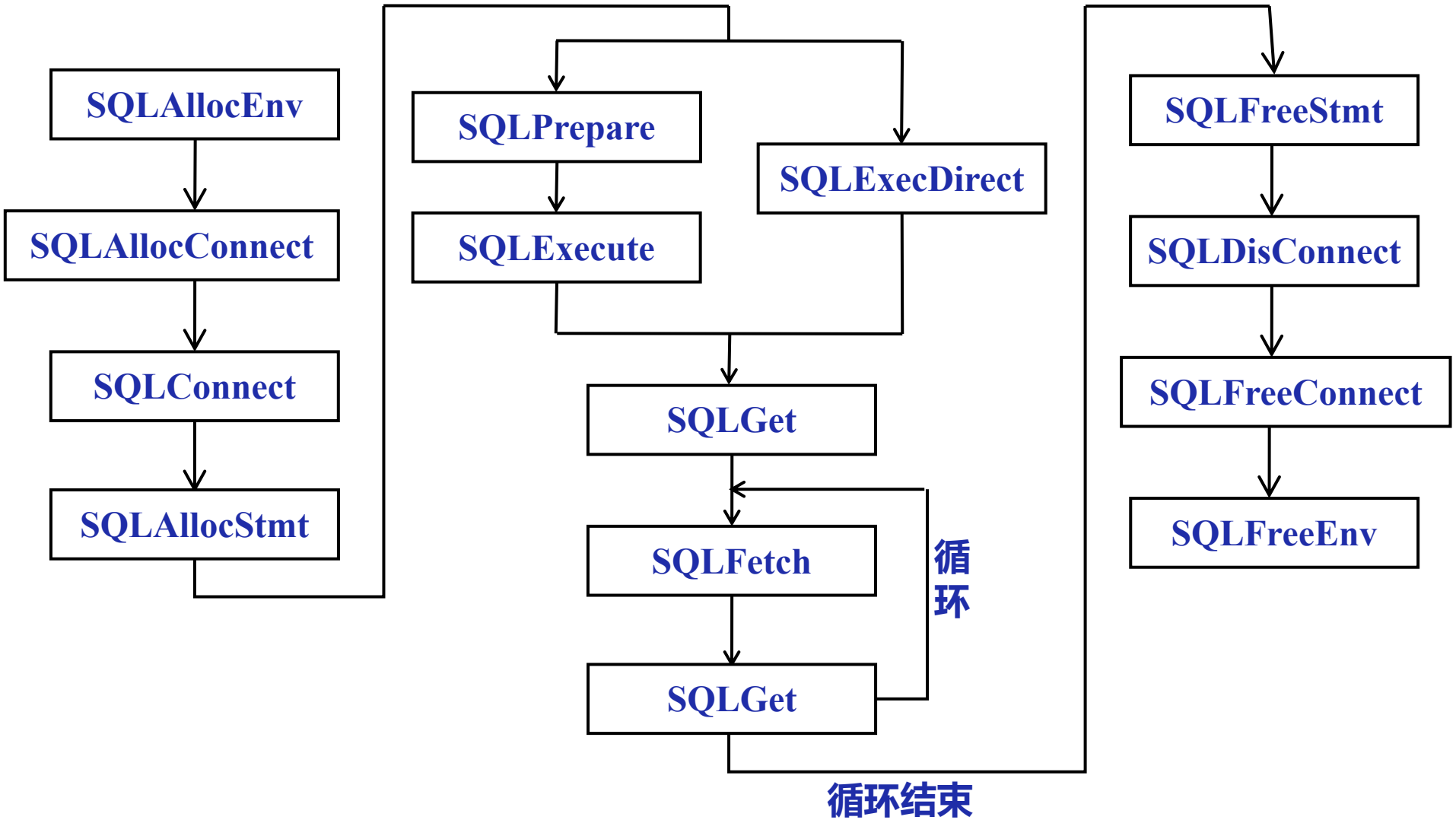


图8.3 ODBC应用系统的体系结构

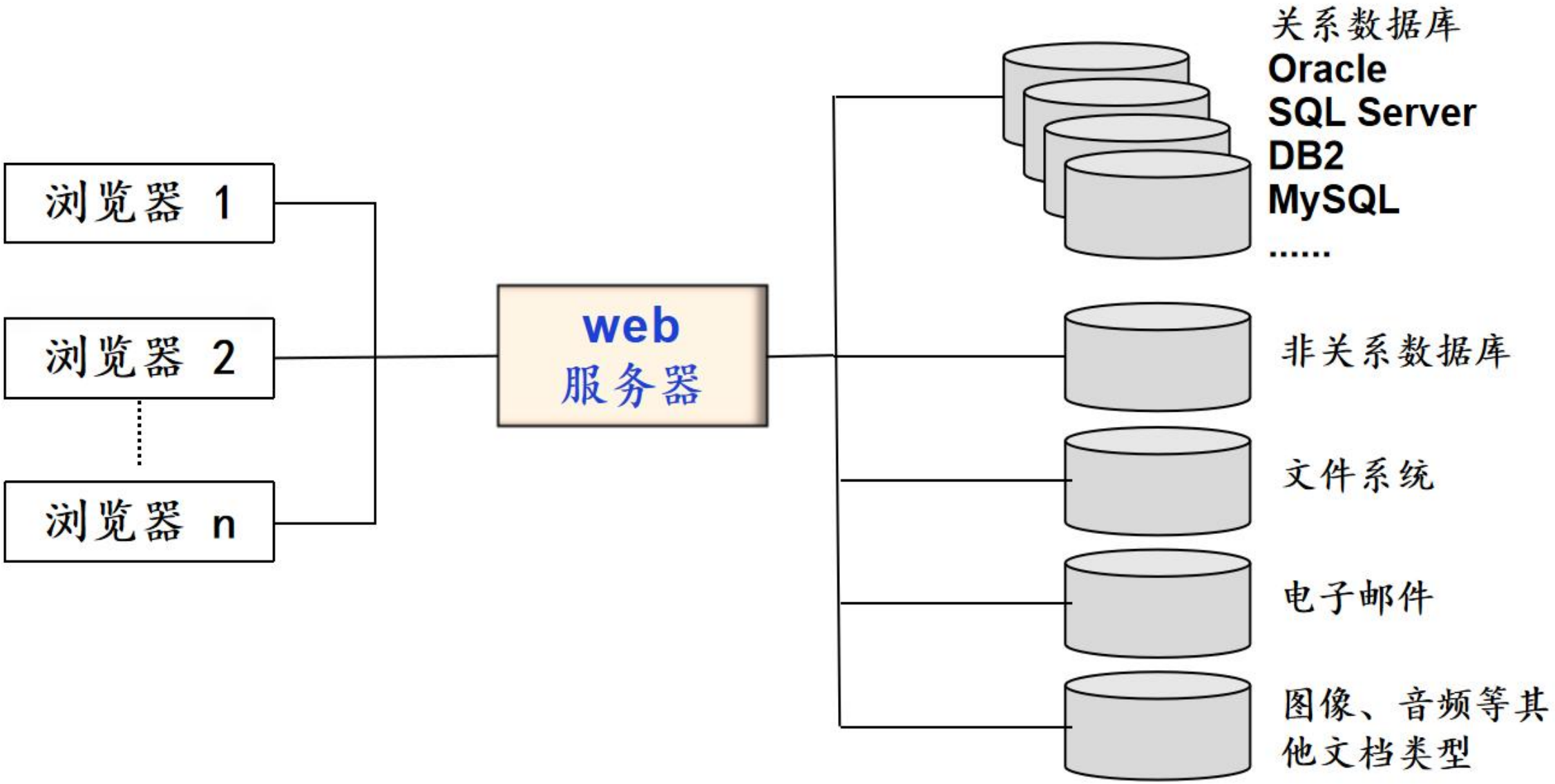
图8.5 ODBC的工作流程



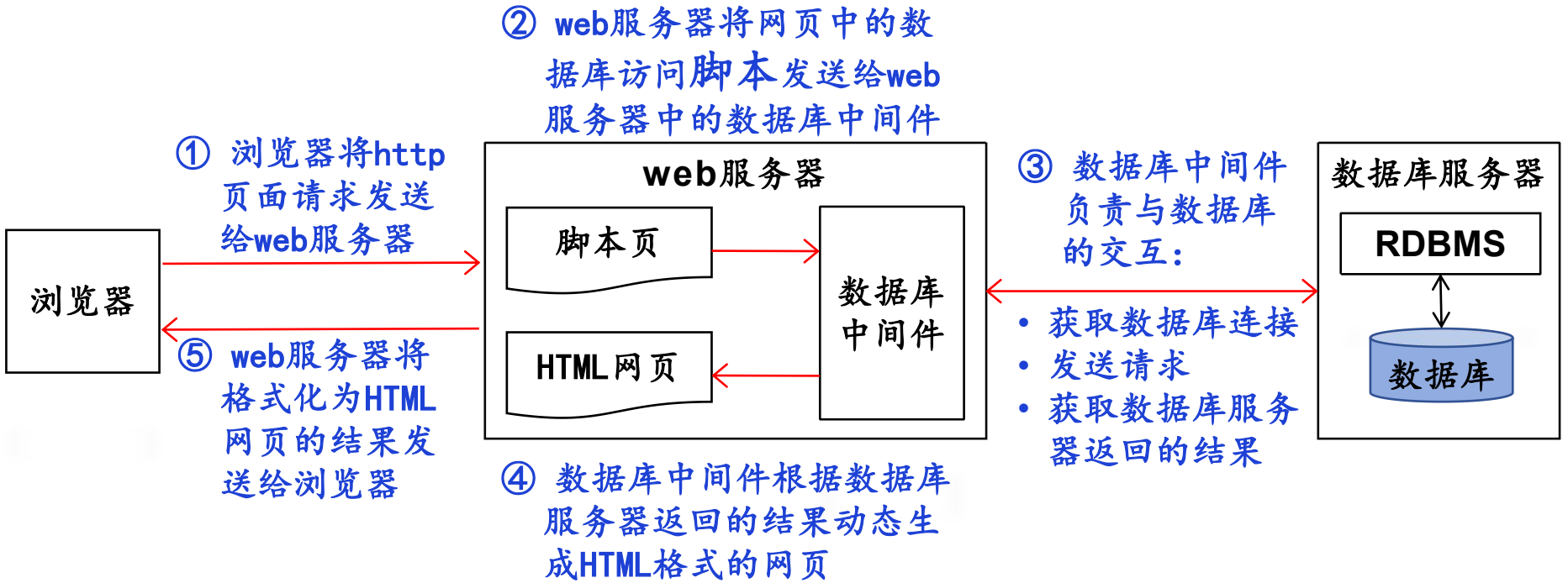
使用ODBC的程序流程



web数据库应用中的各种数据源



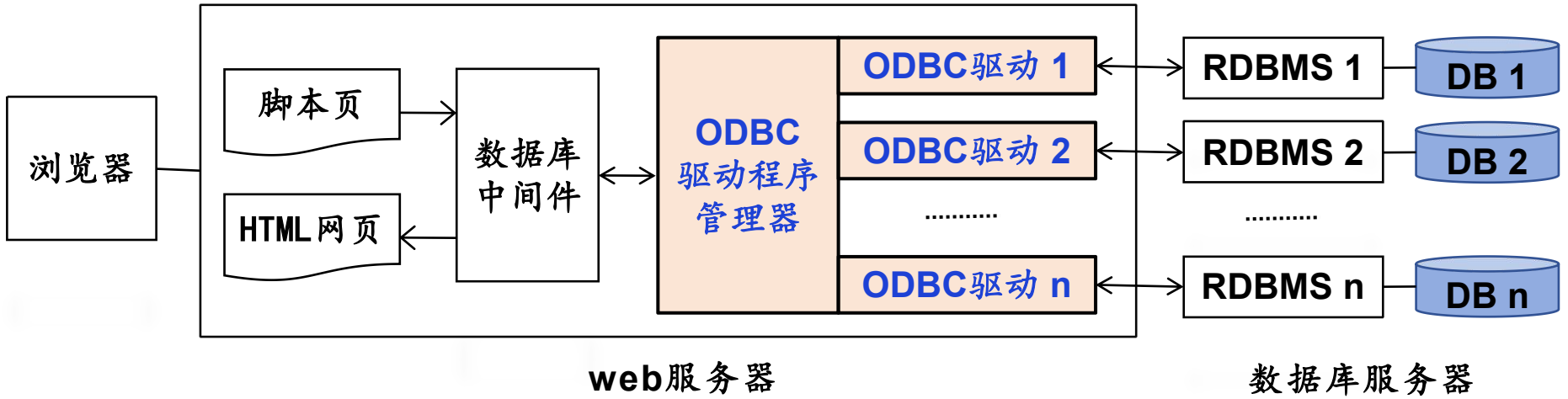
web数据库应用中的数据交换流程



- web服务器中的数据库中间件与数据库服务器的交互（互操作）可以通过两种途径来实现：
  - 使用DBMS厂商提供的专用数据库访问中间件 (如Oracle数据库的OCI、DB2 CLI、MySQL PDO等)
  - 使用通用的数据库访问中间件 (如ODBC、OLE DB、JDBC等)

ODBC体系结构

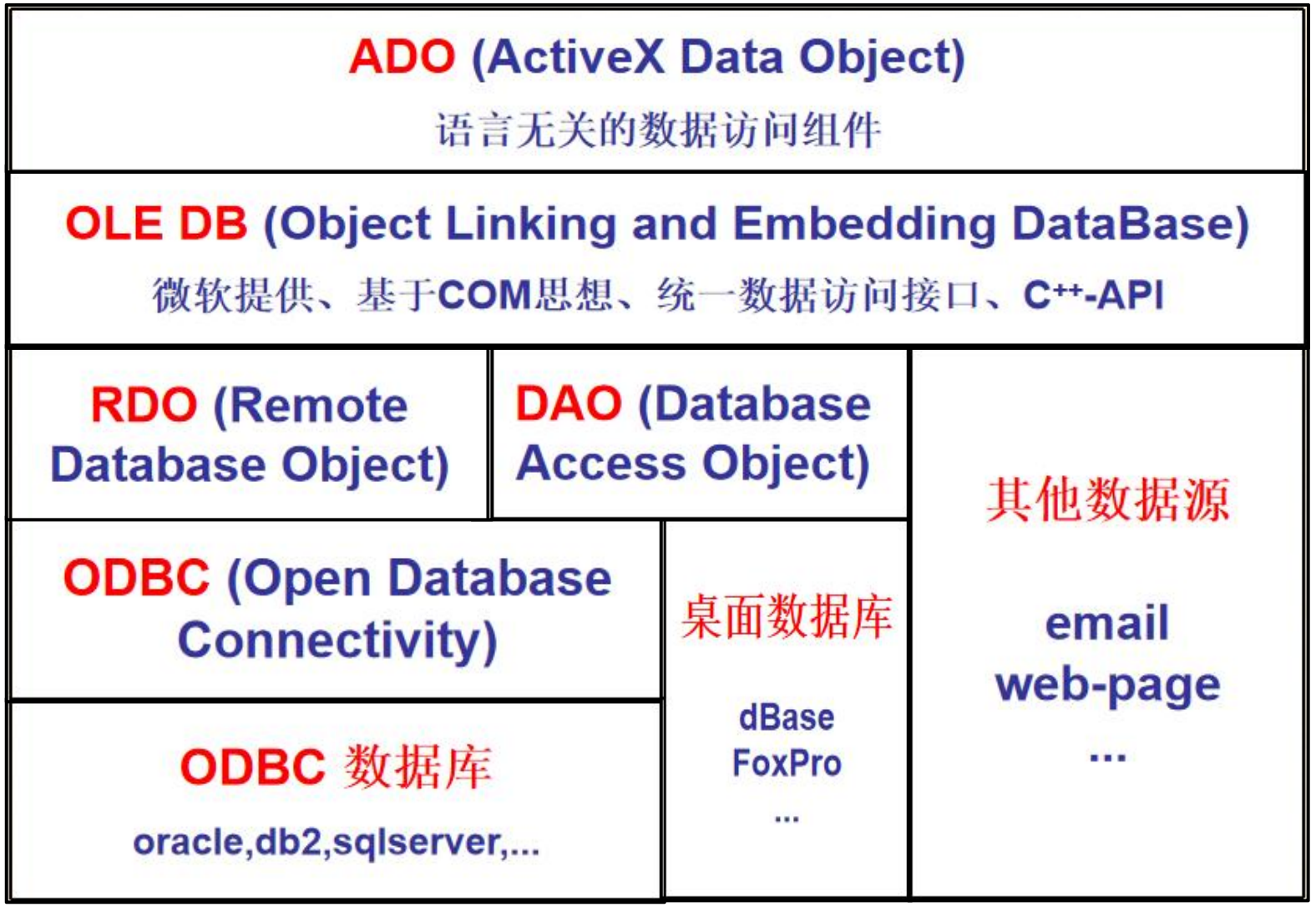
- 开放式数据库连接(Open Database Connectivity，ODBC)是数据库服务器的一个标准协议，它向访问数据库的应用程序提供了一种通用的应用程序访问接口(API)，应用程序开发人员不必知道所连接的数据库平台类型，就可以用标准的SQL语言访问数据库中的数据。
- ODBC通过ODBC驱动程序管理器将用户调用的SQL语句转换成特定数据库的访问函数，不同数据库提供不同的ODBC驱动程序实现。
- 在ODBC客户端，应根据后台数据库类型来选择对应的数据源名称(Data Source Name, DSN)，并指定与后台数据库服务器的连接方式等信息。



Web方式下的数据库网络访问接口

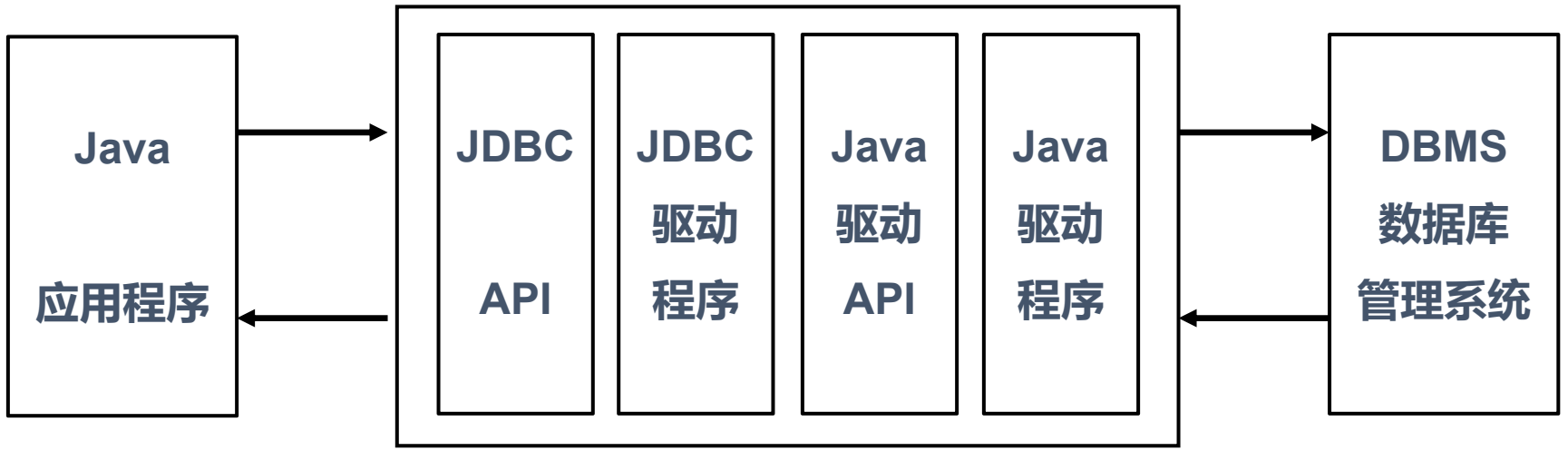
- 常用的数据库网络访问接口
  - **ODBC** (Open Database Connectivity)
    - 基于SQL语言的一种标准化的数据库访问API，统一了不同厂商的关系数据库管理系统的访问接口。
  - **DAO** (Database Access Object)
    - 第一个面向对象的数据库访问接口，既可以访问本地的单机桌面数据库，也可以通过ODBC访问远程数据库。
  - **RDO** (Remote Database Object)
    - 封装了ODBC API的对象层，具有比DAO更高的性能，比ODBC更易用
  - **OLE DB** (Object Linking and Embedding DataBase)
    - 微软提出的基于COM思想的一种技术标准，目的是提供一种统一的数据访问接口（包括关系或非关系型的数据源）
  - **ADO** (ActiveX Data Object)

数据库的网络访问接口

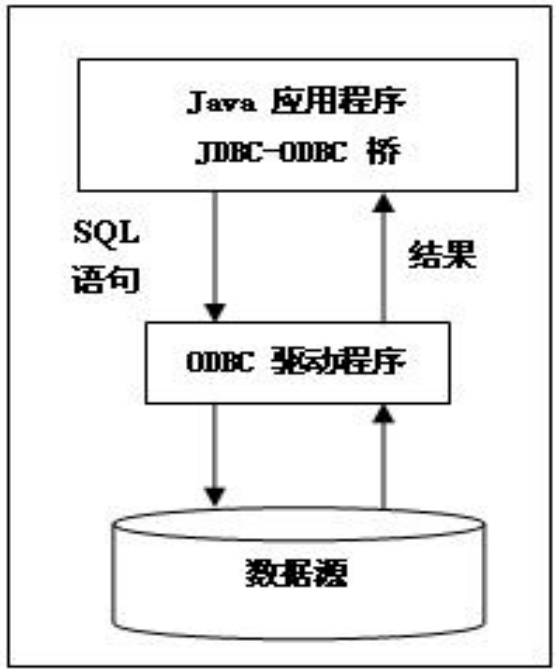

ADO (ActiveX Data Object)
语言无关的数据访问组件
OLE DB (Object Linking and Embedding DataBase)
微软提供、基于COM思想、统一数据访问接口、C++-API
RDO (Remote Database Object)
DAO (Database Access Object)
其他数据源
email
web-page
...
ODBC (Open Database Connectivity)
桌面数据库
dBase
FoxPro
...
ODBC 数据库
oracle,db2,sqlserver,...

使用ODBC的程序流程

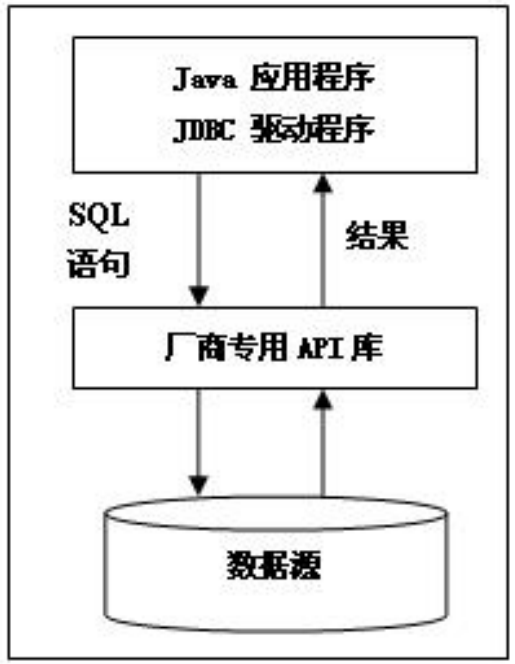
- **JDBC (Java Database Connectivity)**
  - 一种可以执行SQL语句的 Java API



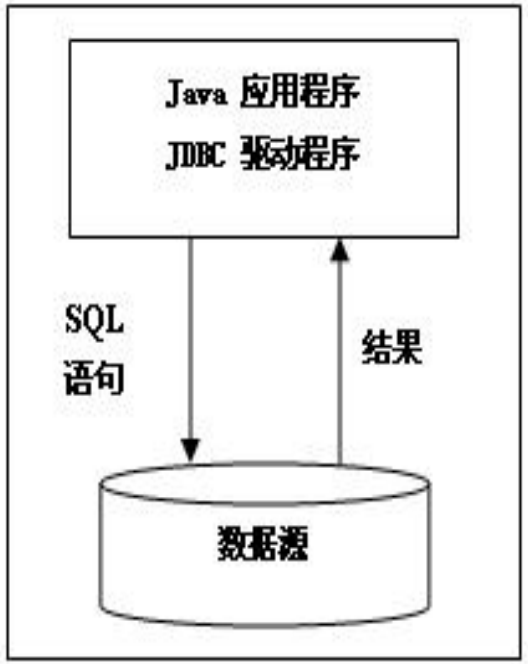
JDBC驱动的实现方式



JDBC-ODBC桥



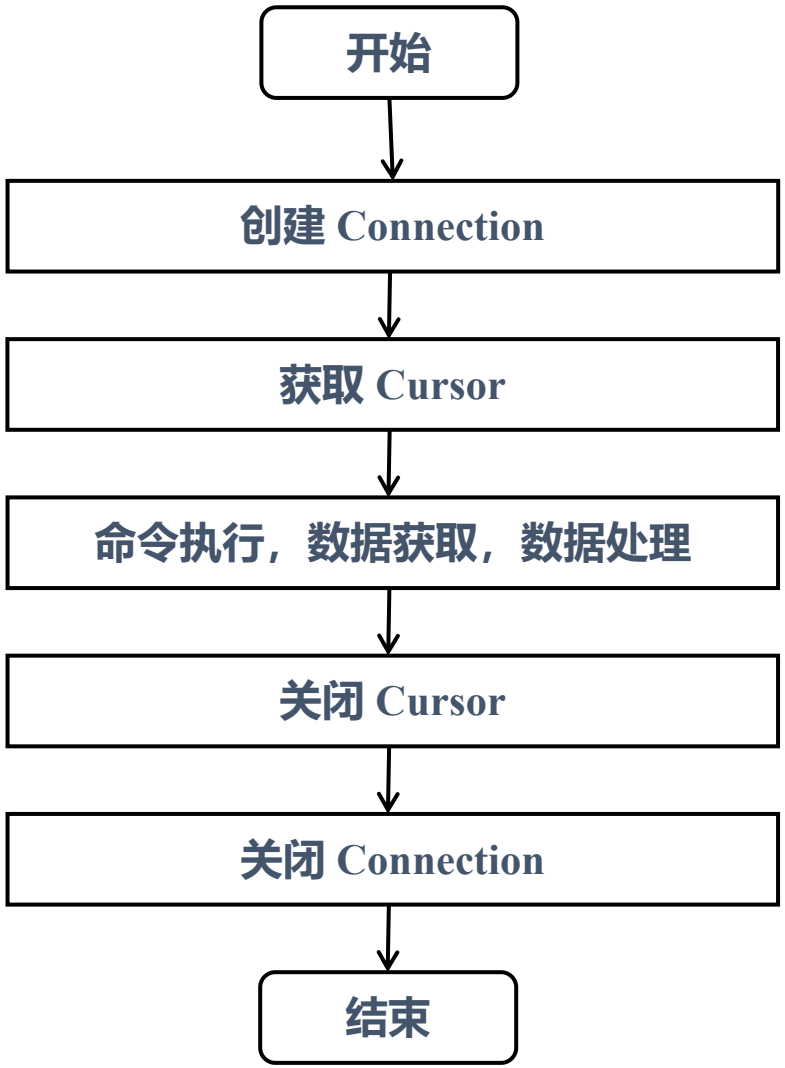
Java+本地驱动



专用协议纯Java驱动

Python程序中的数据库访问流程

(Python DB-API 规范)



复习思考题

1. 使用嵌入式SQL语言编写数据库应用程序，与数据库访问有关的程序构成是什么？
2. 什么是主变量？什么是指示变量？请简述它们的各自作用。
3. 嵌入式SQL与主语言语句之间的信息交换方式有哪些？
4. 什么是游标(cursor)？请简述游标的使用流程。
5. 在使用嵌入式SQL语句访问数据库的过程中，可能会产生哪几种执行异常？

复习思考题

设有一个如下表所示的学生选课数据库，其中：带下划线属性是码；同一门课同一个学生只能有一条选课记录；成绩采用百分制。

关系	属性集	关系模式
学生	学号, 学生姓名, 就读院系, 年级	Student (sno, sname, dept, grade)
课程	课程号, 课程名, 开课院系, 学分	Course (cno, cname, dept, credit)
教师	教师编号, 教师姓名, 工作院系	Teacher (tno, tname, dept)
选课	学号, 课程号, 授课教师编号, 成绩, 授课年份	Scorelist (sno, cno, tno, score, year)

6. 请创建一个存储过程：根据输入的课程号，将该门课程的所有选课成绩从百分制转换成等级制，并插入到临时表temp(sno, cno, scgrade)中去。要求如下：
   ① 有一个输入参数‘课程号’和 两个输出参数‘课程选修人数’、成绩等级为A的学生人数；
   ② 等级转换规则：>=90为A，80~89为B，70~79为C，60~69为D，低于60为E
   ③ 如果成绩为空值，则等级也是空值

7. 请创建一个函数：根据输入的学号，统计该学生的平均成绩（不包括成绩为空值的选课记录）